

UVW 密钥封装机制

算法描述文档

算法提交者:

- 张方国 (中山大学, 计算机学院)
- 杨洋 (江西师范大学, 人工智能学院 (数字产业学院))
- 肖阔杰 (中山大学, 计算机学院)
- 陈晓峰 (西安电子科技大学, 密码学院)
- 李宸 (中山大学, 计算机学院)
- 田海博 (中山大学, 计算机学院)

2026 年 4 月 29 日

目录

1	算法描述	6
1.1	预备知识	6
1.1.1	符号说明	6
1.1.2	编码理论基础	6
1.1.3	Reed-Solomon 码及其列表译码算法	8
1.1.4	公钥加密	10
1.1.5	密钥封装机制	10
1.2	算法工作流程	10
1.2.1	UVW 公钥加密方案	11
1.2.2	UVW 密钥封装机制	14
1.3	算法实例与参数	15
2	设计原理	15
2.1	数学困难问题	15
2.2	失败概率分析及影响	16
2.3	参数选择依据	17
3	安全性声明与分析	18
3.1	理论安全性	18
3.2	具体安全性	22
3.2.1	经典安全强度分析	22
3.2.2	量子安全强度分析	30
4	性能评估	31
4.1	理论性能分析	31
4.2	性能测试	32
5	特点声明	32
	参考文献	34

插图列表

1	Game 0: 加密消息 $\mathbf{m}_0$ 的真实的 IND-CPA 游戏	19
2	Game 1: 随机矩阵 $\mathbf{G}$ 代替公钥矩阵 $\mathbf{G}_{\text{pk}}$	19
3	Game 2: 随机比特串 $\mathbf{s}$ 代替 $\text{Hash}(\mathbf{r}, \mathbf{e})$	20
4	Game 3: 加密消息 $\mathbf{m}_1$ 的真实的 IND-CPA 游戏	21
5	相对重量 w/n 下形式为 $(\mathbf{u}, \mathbf{u})\mathbf{D}$ 的码字密度的归一化对数	25
6	重量为 t 时 $(\mathbf{u}, \mathbf{u})\mathbf{D}$ 码字的期望数量与非 $(\mathbf{u}, \mathbf{u})\mathbf{D}$	29
7	重量 t 在 $[330, 430]$ 时 $(\mathbf{u}, \mathbf{u})\mathbf{D}$ 码字的期望数量与非 $(\mathbf{u}, \mathbf{u})\mathbf{D}$	29

表格列表

1	UVW 密钥封装机制推荐参数集	15
2	UVW 密钥封装机制传输与存储开销（单位：Bytes）	33
3	UVW 密钥封装机制参考实现性能测试结果	33
4	UVW 密钥封装机制优化实现性能测试结果	33

算法列表

1	ListDec.RS($\mathbf{G}, \mathbf{c}, w$): 能够纠正 w 个错误的 Reed-Solomon 列表译码算法	9
2	UVW.PKE.Setup(1^λ): 初始化算法	11
3	UVW.PKE.KeyGen($params$): 密钥生成算法	12
4	UVW.PKE.Enc($pk, \mathbf{m}$): 加密算法	12
5	UVW.PKE.Dec($sk, \mathbf{c}$): 解密算法	13
6	UVW.KEM.Setup(1^λ): 初始化算法	14
7	UVW.KEM.KeyGen($params$): 密钥生成算法	14
8	UVW.KEM.Encap(pk): 密钥封装算法	15
9	UVW.KEM.Decap($sk, (\mathbf{c}_1, \mathbf{c}_2, \mathbf{d})$): 密钥解封装算法	15

1 算法描述

1.1 预备知识

1.1.1 符号说明

$\mathbb{F}_q$ 表示大小为 q 的有限域。粗体小写字母 (例: $\mathbf{x}$) 和大写字母 (例: $\mathbf{H}$) 分别表示行向量和矩阵。 $\mathbf{x}^\top$ ($\mathbf{H}^\top$) 表示向量 $\mathbf{x}$ (矩阵 $\mathbf{H}$) 的转置。令 $m, n > m$ 是正整数, $[n]$ 和 $[m, n]$ 分别表示整数集合 $\{1, 2, \dots, n\}$ 和 $\{m, m+1, \dots, n\}$, S_n 表示 n 阶置换的集合, $\mathcal{DP}(n)$ 表示有限域 $\mathbb{F}_q$ 上 n 阶单项矩阵 (可逆对角矩阵的置换阵) 的全体。 $\mathbf{x} \in \mathbb{F}_q^n$, $\mathbf{x}(i)$ 表示 $\mathbf{x}$ 的第 i 个分量, 有时也记为 x_i , 即 $\mathbf{x} = (x_i)_{i \in [n]}$ 。 $|\mathbf{x}|$ 表示 $\mathbf{x}$ 的汉明重量, 即 $|\mathbf{x}| = |\{i \in [n] \mid \mathbf{x}(i) \neq 0\}|$, 有时也记为 $wt(\mathbf{x})$ 。 $\lfloor a \rfloor$ 表示对有理数 a 向下取整。 $negl(\lambda)$ 表示关于 λ 的可忽略函数。 $\mathcal{E}_{n,w}$ 表示向量空间 $\mathbb{F}_q^n$ 中所有汉明重量为 w 的向量的集合, 集合大小为 $\binom{n}{w}(q-1)^w$ 。 $s \stackrel{\$}{\leftarrow} S$ 表示元素 s 是从集合 S 中均匀随机选择的。令 X 是随机变量。 $\mathbb{P}(X = A)$ 表示事件 A 发生的概率, $\mathbb{E}(X)$ 是它的期望。PRNG 表示伪随机数生成器。

1.1.2 编码理论基础

有限域 $\mathbb{F}_q$ 上的一个 $[n, k]$ 线性码 $\mathcal{C}$ 可以看作是 n 维向量空间 $\mathbb{F}_q^n$ 中的一个 k 维子空间。码 $\mathcal{C}$ 中的向量被称为码字, 其中 n 表示码的长度, k 表示其维数。与用一组基向量表示线性空间类似, 线性码 $\mathcal{C}$ 也可以由码字构成的一组基来表示, 这组基组成的矩阵称为线性码 $\mathcal{C}$ 的生成矩阵。

定义 1 (生成矩阵与校验矩阵). 令 $\mathcal{C}$ 是有限域 $\mathbb{F}_q$ 上的一个 $[n, k]$ 线性码, 则码 $\mathcal{C}$ 的生成矩阵是 $\mathbb{F}_q$ 上的一个 $k \times n$ 的满秩矩阵, 它的行向量构成码 $\mathcal{C}$ 的一组基。如果

$$\mathcal{C} = \{\mathbf{x}\mathbf{G} \mid \mathbf{x} \in \mathbb{F}_q^k\},$$

则 $\mathbf{G}$ 是码 $\mathcal{C}$ 的一个生成矩阵。码 $\mathcal{C}$ 的校验矩阵是 $\mathbb{F}_q$ 上的一个 $(n-k) \times n$ 的满秩矩阵。如果

$$\forall \mathbf{x} \in \mathcal{C} \Leftrightarrow \mathbf{x}\mathbf{H}^\top = \mathbf{0},$$

则 $\mathbf{H}$ 是码 $\mathcal{C}$ 的一个校验矩阵。

码 $\mathcal{C}$ 的生成矩阵和校验矩阵不是唯一的。对于码 $\mathcal{C}$ 的任意生成矩阵 $\mathbf{G}$ 和校验矩阵 $\mathbf{H}$, 都有 $\mathbf{G}\mathbf{H}^\top = \mathbf{0}$ 。一个 $[n, k]$ 线性码 $\mathcal{C}$ 的信息集是 k 个坐标索引的集合, 记作 $I \subset [n]$, 它对应于生成矩阵 $\mathbf{G}$ 中 k 个线性独立列的索引。其补集对应于校验矩阵 $\mathbf{H}$ 中 r ($r = n - k$) 个线性独立列的索引。给定码 $\mathcal{C}$ 的校验矩阵 $\mathbf{H} \in \mathbb{F}_q^{r \times n}$ 以及一个信息集 I , 对于任意向量 $\mathbf{s} \in \mathbb{F}_q^r$ 和 $\mathbf{x} \in \mathbb{F}_q^n$, 存在唯一的向量 $\mathbf{e} \in \mathbb{F}_q^n$ 使得 $\mathbf{e}\mathbf{H}^\top = \mathbf{s}$, 且 $\mathbf{x}_I = \mathbf{e}_I$ 。

定义 2 (($U, U+V$) 码). 有限域 $\mathbb{F}_q$ 上的 $(U, U+V)$ 码由两个线性码 U 和 V 构成。令 n, k_1 和 k_2 是三个正整数, 其中 n 是偶数。令 U 是一个 $[\frac{n}{2}, k_1]$ 线性码, V 是一个 $[\frac{n}{2}, k_2]$ 线性码, 则 $(U, U+V)$ 码的生成矩阵定义如下

$$\mathbf{G} = \begin{pmatrix} \mathbf{G}_U & \mathbf{G}_U \\ \mathbf{0} & \mathbf{G}_V \end{pmatrix},$$

其中 $\mathbf{G}_U$ 和 $\mathbf{G}_V$ 分别是 U 和 V 的生成矩阵。

我们在 $(U, U+V)$ 码的基础上提出了一种 $(U+V, U+W)$ 码, 下面我们给出 $(U+V, U+W)$ 码的正式定义。

定义 3 ($(U+V, U+W)$ 码). 有限域 $\mathbb{F}_q$ 上的 $(U+V, U+W)$ 码由三个线性码 U, V 和 W 构成。令 n, k_1 和 k_2 是三个正整数, 其中 n 是偶数。令 U 是一个 $[\frac{n}{2}, k_1]$ 线性码, V 和 W 都是 $[\frac{n}{2}, k_2]$ 线性码, $\mathbf{G}_U, \mathbf{G}_V$ 和 $\mathbf{G}_W$ 分别是 U, V 和 W 的生成矩阵, 且 $\mathbf{G}_V - \mathbf{G}_W$ 是满秩矩阵。定义如下生成矩阵

$$\mathbf{G} = \begin{pmatrix} \mathbf{G}_U & \mathbf{G}_U \\ \mathbf{G}_V & \mathbf{G}_W \end{pmatrix},$$

则由 $\mathbf{G}$ 生成的 $[n, k]$ 线性码 ($k = k_1 + k_2$), 称为 $(U+V, U+W)$ 码。

下面, 我们给出广义 $(U+V, U+W)$ 码的定义, 这也是本文加密方案所使用的码。

定义 4 (广义 $(U+V, U+W)$ 码). 有限域 $\mathbb{F}_q$ 上的 $(U+V, U+W)$ 码由三个线性码 U, V 和 W 构成。令 n, k_1 和 k_2 是三个正整数, 其中 n 是偶数。令 U 是一个 $[\frac{n}{2}, k_1]$ 线性码, V 和 W 都是 $[\frac{n}{2}, k_2]$ 线性码, $\mathbf{D} \in \mathbb{F}_q^{n \times n}$ 是随机单项矩阵, $\mathbf{G}_U, \mathbf{G}_V$ 和 $\mathbf{G}_W$ 分别是 U, V 和 W 的生成矩阵, 且 $\mathbf{G}_V - \mathbf{G}_W$ 是满秩矩阵。定义如下生成矩阵

$$\mathbf{G} = \begin{pmatrix} \mathbf{G}_U & \mathbf{G}_U \\ \mathbf{G}_V & \mathbf{G}_W \end{pmatrix} \mathbf{D},$$

则由 $\mathbf{G}$ 生成的 $[n, k]$ 线性码 ($k = k_1 + k_2$), 称为广义 $(U+V, U+W)$ 码。

推论 1. 令 $\mathcal{C}$ 是一个广义 $[n, k]$ $(U+V, U+W)$ 码, 其生成矩阵为 $\mathbf{G} = \begin{pmatrix} \mathbf{G}_U & \mathbf{G}_U \\ \mathbf{G}_V & \mathbf{G}_W \end{pmatrix} \mathbf{D}$, 定义其对偶码 $\mathcal{C}^\perp$ 为

$$\mathcal{C}^\perp = \{\mathbf{y} \in \mathbb{F}_q^n \mid \forall \mathbf{x} \in \mathcal{C}, \langle \mathbf{x}, \mathbf{y} \rangle = 0\},$$

则码 $\mathcal{C}^\perp$ 是一个广义 $[n, n-k]$ $(U+V, U+W)$ 码。

证明. 推论 1 的证明如下。由定义 4 可知, $\mathbf{G}_U, \mathbf{G}_V$ 和 $\mathbf{G}_W$ 分别是 U, V 和 W 的生成矩阵, 且 $\mathbf{G}_\Delta = \mathbf{G}_W - \mathbf{G}_V$ 行满秩。为了构造 $\mathcal{C}$ 的校验矩阵 $\mathbf{H}$ (即对偶码 $\mathcal{C}^\perp$ 的生成矩阵), 我们直接基于底层子码的对偶关系定义以下三个矩阵: 1) 令 $\mathbf{H}_U \in \mathbb{F}_q^{(n/2-k_1) \times n/2}$ 为子码 U 的校验矩阵, 满足 $\mathbf{G}_U \mathbf{H}_U^T = \mathbf{0}$; 2) 令 $\mathbf{H}_\Delta \in \mathbb{F}_q^{(n/2-k_2) \times n/2}$ 为由 $\mathbf{G}_\Delta$ 生成的子码的校验矩阵, 满足 $(\mathbf{G}_W - \mathbf{G}_V) \mathbf{H}_\Delta^T = \mathbf{0}$ 。3) 由于 $\mathbf{G}_\Delta$ 行满秩, 非齐次线性方程组 $\mathbf{G}_\Delta \mathbf{Y}^T = -\mathbf{G}_V \mathbf{H}_U^T$ 存在无穷多解。依托解空间的自由度, 我们可以从中选取一个特定的特解矩阵记为 $\mathbf{H}_Y \in \mathbb{F}_q^{(n/2-k_1) \times n/2}$, 使得 $\mathbf{H}_Y$ 本身满秩, 且导出的矩阵差 $\mathbf{H}_U - \mathbf{H}_Y$ 也为满秩矩阵。

基于上述组件, 我们构造候选校验矩阵 $\mathbf{H}$ 如下:

$$\mathbf{H} = \begin{pmatrix} \mathbf{H}_U - \mathbf{H}_Y & \mathbf{H}_Y \\ -\mathbf{H}_\Delta & \mathbf{H}_\Delta \end{pmatrix} (\mathbf{D}^{-1})^T,$$

我们需要验证 $\mathbf{G} \mathbf{H}^T = \mathbf{0}$ 。展开矩阵乘法:

$$\mathbf{G} \mathbf{H}^T = \begin{pmatrix} \mathbf{G}_U & \mathbf{G}_U \\ \mathbf{G}_V & \mathbf{G}_W \end{pmatrix} \mathbf{D} \cdot \mathbf{D}^{-1} \begin{pmatrix} (\mathbf{H}_U - \mathbf{H}_Y)^T & -\mathbf{H}_\Delta^T \\ \mathbf{H}_Y^T & \mathbf{H}_\Delta^T \end{pmatrix}$$

$$= \begin{pmatrix} \mathbf{G}_U & \mathbf{G}_U \\ \mathbf{G}_V & \mathbf{G}_W \end{pmatrix} \begin{pmatrix} \mathbf{H}_U^T - \mathbf{H}_Y^T & -\mathbf{H}_\Delta^T \\ \mathbf{H}_Y^T & \mathbf{H}_\Delta^T \end{pmatrix},$$

分别计算结果矩阵的四个分块：

1. 左上块: $\mathbf{G}_U(\mathbf{H}_U^T - \mathbf{H}_Y^T) + \mathbf{G}_U\mathbf{H}_Y^T = \mathbf{G}_U\mathbf{H}_U^T - \mathbf{G}_U\mathbf{H}_Y^T + \mathbf{G}_U\mathbf{H}_Y^T = \mathbf{G}_U\mathbf{H}_U^T = \mathbf{0}$;
2. 右上块: $-\mathbf{G}_U\mathbf{H}_\Delta^T + \mathbf{G}_U\mathbf{H}_\Delta^T = \mathbf{0}$;
3. 右下块: $-\mathbf{G}_V\mathbf{H}_\Delta^T + \mathbf{G}_W\mathbf{H}_\Delta^T = \mathbf{G}_\Delta\mathbf{H}_\Delta^T = \mathbf{0}$;
4. 左下块: $\mathbf{G}_V(\mathbf{H}_U^T - \mathbf{H}_Y^T) + \mathbf{G}_W\mathbf{H}_Y^T = \mathbf{G}_V\mathbf{H}_U^T + (\mathbf{G}_W - \mathbf{G}_V)\mathbf{H}_Y^T$, 代入 $\mathbf{H}_Y$ 的定义式 $(\mathbf{G}_W - \mathbf{G}_V)\mathbf{H}_Y^T = -\mathbf{G}_V\mathbf{H}_U^T$, 可得 $\mathbf{G}_V\mathbf{H}_U^T - \mathbf{G}_V\mathbf{H}_U^T = \mathbf{0}$ 。至此, $\mathbf{GH}^T = \mathbf{0}$ 得到严格证明。

接下来验证校验矩阵 $\mathbf{H}$ 的秩。考察 $\mathbf{H}$ 的核心分块矩阵 $\mathbf{M} = \begin{pmatrix} \mathbf{H}_U - \mathbf{H}_Y & \mathbf{H}_Y \\ -\mathbf{H}_\Delta & \mathbf{H}_\Delta \end{pmatrix}$, 我们将 $\mathbf{M}$ 的右侧列块加到左侧列块上 (初等列变换不改变矩阵的秩), 可得:

$$\mathbf{M} \xrightarrow{C_1 \leftarrow C_1 + C_2} \begin{pmatrix} (\mathbf{H}_U - \mathbf{H}_Y) + \mathbf{H}_Y & \mathbf{H}_Y \\ -\mathbf{H}_\Delta + \mathbf{H}_\Delta & \mathbf{H}_\Delta \end{pmatrix} = \begin{pmatrix} \mathbf{H}_U & \mathbf{H}_Y \\ \mathbf{0} & \mathbf{H}_\Delta \end{pmatrix},$$

这转化为了一个分块上三角矩阵, 该矩阵的秩为其对角分块的秩之和:

$$\text{Rank}(\mathbf{M}) = \text{Rank}(\mathbf{H}_U) + \text{Rank}(\mathbf{H}_\Delta) = (n/2 - k_1) + (n/2 - k_2) = n - (k_1 + k_2) = n - k,$$

由于 $(\mathbf{D}^{-1})^T$ 是可逆矩阵, 右乘不改变矩阵的秩, 故 $\mathbf{H} = \mathbf{M}(\mathbf{D}^{-1})^T$ 具有满秩 $n - k$ 。

调整矩阵 $\mathbf{H}$ 的分块行顺序, 并吸收负号 (对角置换), 即可将其化为标准广义 $(U + V, U + W)$ 码生成矩阵的形式。因此, 对偶码 $\mathcal{C}^\perp$ 也是广义 $(U + V, U + W)$ 码。证毕。 $\square$

我们定义 $H_X = H_U - H_Y$ 以及 $H_Z = H_\Delta$, 同时可知 $(\mathbf{D}^{-1})^T$ 仍为单项矩阵。此时, 校验矩阵可等价重写为:

$$\mathbf{H} = \begin{pmatrix} \mathbf{H}_X & \mathbf{H}_Y \\ -\mathbf{H}_Z & \mathbf{H}_Z \end{pmatrix} \mathbf{D}$$

由定义 1 可知, 任意的线性码都能被生成矩阵 $\mathbf{G}$ 或校验矩阵 $\mathbf{H}$ 表示。为方便起见, 除非特别说明, 我们将用如下定义的生成矩阵

$$\mathbf{G} = \begin{pmatrix} \mathbf{G}_U & \mathbf{G}_U \\ \mathbf{G}_V & \mathbf{G}_W \end{pmatrix} \mathbf{D},$$

表示一个广义 $[n, k]$ $(U + V, U + W)$ 码, 其中 $\mathbf{G}_U \in \mathbb{F}_q^{k_1 \times n/2}$, $\mathbf{G}_V, \mathbf{G}_W \in \mathbb{F}_q^{k_2 \times n/2}$, $k = k_1 + k_2$ 。

1.1.3 Reed-Solomon 码及其列表译码算法

定义 5 (Reed-Solomon 码). 令 $\vec{\alpha} = (\alpha_1, \alpha_2, \dots, \alpha_n)$, α_i ($i \in [n]$) 是 $\mathbb{F}_q$ 中的不同的非零元素。则由 $\vec{\alpha}$ 定义的 k 维 Reed-Solomon 码 $\mathcal{C}$ 为

$$\mathcal{C} = \{(f(\alpha_1), f(\alpha_2), \dots, f(\alpha_n)) \mid \deg(f) < k\}.$$

令 $f_j(x) = x^j$ ($j = 0 \cdots, k-1$), 则矩阵

$$\mathbf{G} = \begin{pmatrix} f_0(\alpha_1) & f_0(\alpha_2) & \cdots & f_0(\alpha_n) \\ f_1(\alpha_1) & f_1(\alpha_2) & \cdots & f_1(\alpha_n) \\ \vdots & \vdots & \ddots & \vdots \\ f_{k-1}(\alpha_1) & f_{k-1}(\alpha_2) & \cdots & f_{k-1}(\alpha_n) \end{pmatrix} = \begin{pmatrix} 1 & 1 & \cdots & 1 \\ \alpha_1 & \alpha_2 & \cdots & \alpha_n \\ \vdots & \vdots & \ddots & \vdots \\ \alpha_1^{k-1} & \alpha_2^{k-1} & \cdots & \alpha_n^{k-1} \end{pmatrix}$$

是码 $\mathcal{C}$ 的一个生成矩阵。码 $\mathcal{C}$ 的对偶码 $\mathcal{C}^\perp$ 是一个 $[n, n-k]$ Reed-Solomon 码。Reed-Solomon 码也可由生成多项式 $g(x)$ 定义。令 $\alpha \in \mathbb{F}_q$ 是本原域元素, 如果要求码的最小距离为 d , 则其生成多项式可以表示为 $g(x) = \prod_{i=1}^{d-1} (x - \alpha^i)$, 其校验矩阵定义如下

$$\mathbf{H} = \begin{pmatrix} \alpha^{n-1} & \alpha^{n-2} & \cdots & \alpha^2 & \alpha & 1 \\ (\alpha^2)^{n-1} & (\alpha^2)^{n-2} & \cdots & (\alpha^2)^2 & (\alpha^2) & 1 \\ \vdots & \vdots & & \vdots & \vdots & \vdots \\ (\alpha^{d-1})^{n-1} & (\alpha^{d-1})^{n-2} & \cdots & (\alpha^{d-1})^2 & (\alpha^{d-1}) & 1 \end{pmatrix}.$$

Reed-Solomon 码是极大距离可分码, 它的最小距离也就是最小重量码字的重量为 $d = n - k + 1$ 。列表译码算法能够纠正超过最小距离一半以上的错误, 下面给出 Reed-Solomon 码的 Guruswami-Sudan 列表译码算法 [1]。关于 Guruswami-Sudan 算法中译码参数的选取, 本文参考了 Wang [2] 提出的改进参数选取方案, 以获得更好的译码性能。

算法 1 ListDec.RS($\mathbf{G}, \mathbf{c}, w$): 能够纠正 w 个错误的 Reed-Solomon 列表译码算法

输入: RS 码生成矩阵 $\mathbf{G} \in \mathbb{F}_q^{k \times n}$, 接收向量 $\mathbf{c} \in \mathbb{F}_q^n$, 译码半径 w

- 1: 从 $\mathbf{G}$ 的结构中提取支撑集 $\mathbf{x} = (x_1, \cdots, x_n)$
- 2: 根据 w, n, k 计算 Guruswami-Sudan 译码参数: 插值重数 M 和 y 的最高维度 L
- 3: **插值步 (Interpolation):** 寻找一个非零二元多项式 $Q(x, y) \in \mathbb{F}_q[x, y]$, 满足具有最小的 $(1, k-1)$ -加权重数, 且对于所有的 $i \in \{1, \cdots, n\}$, 点 (x_i, c_i) 的重数至少为 M
- 4: **因子分解步 (Factorization):** 计算 $Q(x, y)$ 满足度数限制的所有一元多项式根, 得到候选消息多项式集合:

$$\mathcal{F} = \{f(x) \in \mathbb{F}_q[x] \mid Q(x, f(x)) = 0, \deg(f) < k\}$$

- 5: 初始化有效输出集合 $\mathcal{V} = \emptyset$
- 6: **筛选步 (Selection):** 对于 $\mathcal{F}$ 中的每一个 $f(x)$:
- 7: 提取 $f(x)$ 的系数, 构造候选消息向量 $\mathbf{r} = (f_0, f_1, \cdots, f_{k-1}) \in \mathbb{F}_q^k$
- 8: 计算对应的错误向量 $\mathbf{e} = \mathbf{c} - \mathbf{r}\mathbf{G}$
- 9: 如果其 Hamming 重量 $\text{wt}(\mathbf{e}) \leq w$, 则令 $\mathcal{V} = \mathcal{V} \cup \{(\mathbf{r}, \mathbf{e})\}$

10: **return** $\mathcal{V}$

输出: 满足条件的“消息-错误”对集合 $\mathcal{V} = \{(\mathbf{r}, \mathbf{e})\}$

1.1.4 公钥加密

定义 6 (公钥加密). 一个典型的公钥加密方案主要由以下三个多项式时间算法构成: KeyGen、Encrypt 和 Decrypt。

- KeyGen(1^λ): KeyGen(1^λ) 是一个概率多项式时间算法, 根据输入的系统安全参数 1^λ , 输出一对公私钥对 (pk, sk) 。记作: $(pk, sk) \leftarrow \text{KeyGen}(1^\lambda)$, 其中 pk 是公钥, 公开给所有人, 用于加密消息; 而 sk 是私钥, 仅为持有者所知, 用于解密消息;
- Encrypt(pk, m): Encrypt(pk, m) 是一个概率多项式时间算法, 输入消息 m 和公钥 pk , 输出消息 m 对应的密文 c 。记作: $c \leftarrow \text{Encrypt}(pk, m)$;
- Decrypt(sk, c): Decrypt(sk, c) 是一个确定性多项式时间算法, 输入接收到的密文 c 和私钥 sk , 输出密文 c 对应的明文消息 m , 如果密文 c 是不合法的, 则输出终止符 $\perp$ 表示解密失败。记作: $m = \text{Decrypt}(sk, c)$ 。

正确性: 对任意的公私钥对 $(pk, sk) \leftarrow \text{KeyGen}(1^\lambda)$, 任意的消息 m , 以下等式总是成立:

$$\text{Decrypt}(sk, \text{Encrypt}(pk, m)) = m. \quad (1)$$

这意味着, 在给定公钥和私钥的情况下, 使用加密算法加密的消息必须能够通过解密算法准确还原为原始消息。换言之, 对于任意消息 m , 加密后生成的密文 c 在经过解密后, 必须还原回原始消息 m 。

1.1.5 密钥封装机制

定义 7 (密钥封装). 一个密钥封装方案主要由以下三个多项式时间算法构成: KeyGen、Encaps 和 Decaps。

- KeyGen(1^λ): KeyGen(1^λ) 是一个概率多项式时间算法, 根据输入的系统安全参数 1^λ , 输出一对公私钥对 (pk, sk) 。记作: $(pk, sk) \leftarrow \text{KeyGen}(1^\lambda)$, 其中 pk 是公钥, 公开给所有人, 用于加密消息; 而 sk 是私钥, 仅为持有者所知, 用于解密消息;
- Encaps($pk, 1^\lambda$): Encaps($pk, 1^\lambda$) 是一个概率多项式时间算法, 输入公钥 pk 和安全参数 1^λ , 输出密文 c 和会话密钥 K 。记作: $(c, K) \leftarrow \text{Encaps}(pk, 1^\lambda)$;
- Decaps(sk, c): Decaps(sk, c) 是一个确定性多项式时间算法, 输入私钥 sk 和接收到的密文 c , 输出会话密钥 K 或者终止符 $\perp$ 表示解密失败。记作: $K = \text{Decaps}(sk, c)$ 。

1.2 算法工作流程

在这一节中, 我们将详细描述 UVW 密钥封装方案的具体构造, 为了便于理解, 我们将先给出选择明文攻击安全的 UVW 公钥加密方案的具体构造。

1.2.1 UVW 公钥加密方案

本小节我们给出 UVW 公钥加密方案的具体构造。UVW 公钥加密方案是基于广义 $(U+V, U+W)$ 码构造的, 该方案包含以下四个算法: Setup, KeyGen, Enc 和 Dec。

在密钥生成算法中我们引入计数器 i_1, i_2, i_3 并将其与种子 $seed$ 共同保存为私钥, 核心目的是让解密方能够一次性、确定性地重构与加密方完全相同的矩阵, 从而省去密钥生成阶段那些依赖随机试探的验证步骤, 特别是矩阵满秩判断, 从而加快解密速度。具体而言, 各矩阵均采用种子拼接计数器与固定标签, 然后通过 PRNG 扩展生成随机字节, 每次取 2 字节组装为 16 位整数, 取其低 9 位作为候选值 (在 UVW128 参数下); 若候选值小于域大小 q 则接受为 $\mathbb{F}_q$ 的一个元素, 否则重试。对于 $\mathbf{G}_U, \mathbf{G}_W$, 所得元素按行优先依次填入 $k_1 \times n/2$ 或 $k_2 \times n/2$ 矩阵。因解密方直接使用保存的计数器值, 故无需再执行满秩检查, 可直接获得与加密方完全相同的满秩矩阵。对于单项矩阵 $\mathbf{D}$, 生成长度为 n 的随机置换和长度为 n 的非零随机系数数组来共同存储。

对于有限域 $\mathbb{F}_q$ 上生成的随机矩阵, 一个大小为 $r \times c$ ($r < c$) 的矩阵满秩的概率为 $\prod_{i=0}^{r-1} (1 - q^{i-c})$ [3], 其不满秩的概率上界约为 $q^{-(c-r+1)}$ 。在密钥生成算法中, 假设 $(q, n, k_1, k_2) = (433, 860, 215, 215)$, 则随机生成的矩阵 $\mathbf{G}_U, \mathbf{G}_W$ 不满秩的概率约为 433^{-216} , 这意味着满秩矩阵 $\mathbf{G}_U, \mathbf{G}_W$ 的生成几乎必然在第一次尝试时成功。

另一方面, 密钥生成算法中我们需要寻找单项矩阵 $\mathbf{D}$ 使得 $\mathbf{G}_{sk} = (\mathbf{A} \parallel \mathbf{B})$ 能够通过行变换化为系统形式, 这等价于判断一个大小为 $k \times k$ 的随机矩阵 $\mathbf{A}$ 是否可逆, 若 $\mathbf{A}$ 可逆意味着我们找到了合适的 $\mathbf{D}$ 。在 $\mathbb{F}_q$ 上随机选取一个 $n \times n$ 方阵不可逆的概率为 $1 - \prod_{j=1}^n (1 - q^{-j})$ [3], 其上界为 $\frac{1}{q-1}$, 假设 $(q, n, k) = (433, 860, 430)$, 则 $\mathbf{A}$ 不可逆的概率约为 0.0023, 这意味着几乎只需一次尝试即可找到满足条件的 $\mathbf{D}$ 来获得系统形式矩阵。

算法 2 UVW.PKE.Setup(1^λ): 初始化算法

输入: 安全参数 λ

- 1: 根据安全参数选择相应的系统参数 $params = (q, n, k, k_1, k_2, w, m)$, 其中 $k = k_1 + k_2$
 - 2: 选择一个安全的哈希函数 $\mathcal{H}: \{0, 1\}^* \rightarrow \mathbb{F}_2^m$
 - 3: 定义有限域 $\mathbb{F}_q$, 之后的所有运算都在有限域 $\mathbb{F}_q$ 上进行
-

算法 3 UVW.PKE.KeyGen($params$): 密钥生成算法

输入: 系统参数 $params$

- 1: 随机生成种子 $seed \in \{0,1\}^m$; 生成计数器 $i_1, i_2, i_3 \in \{0,1\}^8$, 均初始化为 0
- 2: 令 $i_1 = i_1 + 1$, 生成矩阵 $\mathbf{G}_U \leftarrow \text{PRNG}(seed \parallel i_1 \parallel 1) \in \mathbb{F}_q^{k_1 \times n/2}$, 若 $\mathbf{G}_U$ 不满秩, 则重复此步
- 3: 令 $i_2 = i_2 + 1$, 生成矩阵 $\mathbf{G}_W \leftarrow \text{PRNG}(seed \parallel i_2 \parallel 2) \in \mathbb{F}_q^{k_2 \times n/2}$, 若 $\mathbf{G}_W$ 不满秩, 则重复此步
- 4: 令 $\mathbf{G}_{RS} \in \mathbb{F}_q^{k_2 \times n/2}$ 是有限域 $\mathbb{F}_q$ 上能纠 w 个错误, 且由 $\vec{\alpha} = (1, 2, \dots, n/2)$ 定义的 $[n/2, k_2]$ Reed-Solomon 码 $\mathcal{C}$ 的生成矩阵
- 5: 计算 $\mathbf{G}_V = \mathbf{G}_{RS} + \mathbf{G}_W$
- 6: 令 $i_3 = i_3 + 1$, 生成单项矩阵 $\mathbf{D} \leftarrow \text{PRNG}(seed \parallel i_3 \parallel 3) \in \mathcal{DP}(n)$
- 7: 计算 $\mathbf{G}_{sk} = \begin{pmatrix} \mathbf{G}_U & \mathbf{G}_U \\ \mathbf{G}_V & \mathbf{G}_W \end{pmatrix} \mathbf{D}$
- 8: 对 $\mathbf{G}_{sk}$ 进行初等行变换, 尝试将其化为系统形式 $(\mathbf{I}_k \parallel \mathbf{T})$, 若失败, 则返回第 6 步, 重新生成单项矩阵 $\mathbf{D}$

输出: 私钥 $sk = (seed, i_1, i_2, i_3)$, 公钥 $pk = \mathbf{T}$

算法 4 UVW.PKE.Enc($pk, \mathbf{m}$): 加密算法

输入: 公钥 $pk = \mathbf{T}$ 和待加密的消息 $\mathbf{m} \in \mathbb{F}_2^m$

- 1: 随机选择向量 $\mathbf{r} \xleftarrow{\$} \mathbb{F}_q^k$
- 2: 随机选择一个重量为 w 的向量 $\mathbf{e} \in \mathcal{E}_{q,n}^w$
- 3: 令 $\mathbf{G}_{pk} = (\mathbf{I}_k \parallel \mathbf{T})$
- 4: 计算 $\mathbf{c}_1 = \mathbf{r} \mathbf{G}_{pk} + \mathbf{e}$, $\mathbf{c}_2 = \mathbf{m} \oplus \mathcal{H}(\mathbf{r}, \mathbf{e})$

输出: 密文 $\mathbf{c} = (\mathbf{c}_1, \mathbf{c}_2)$

算法 5 UVW.PKE.Dec(sk, c): 解密算法

输入: 私钥 $\text{sk} = (\text{seed}, i_1, i_2, i_3)$, 密文 $\mathbf{c} = (\mathbf{c}_1, \mathbf{c}_2)$

- 1: 重构满秩矩阵 $\mathbf{G}_U \leftarrow \text{PRNG}(\text{seed} \parallel i_1 \parallel 1) \in \mathbb{F}_q^{k_1 \times n/2}$
- 2: 重构满秩矩阵 $\mathbf{G}_W \leftarrow \text{PRNG}(\text{seed} \parallel i_2 \parallel 2) \in \mathbb{F}_q^{k_2 \times n/2}$
- 3: 重构单项矩阵 $\mathbf{D} \leftarrow \text{PRNG}(\text{seed} \parallel i_3 \parallel 3) \in \mathcal{DP}(n)$
- 4: 令 $\mathbf{G}_{RS} \in \mathbb{F}_q^{k_2 \times n/2}$ 是有限域 $\mathbb{F}_q$ 上能纠 w 个错误, 且由 $\vec{\alpha} = (1, 2, \dots, n/2)$ 定义的 $[n/2, k_2]$ Reed-Solomon 码 $\mathcal{C}$ 的生成矩阵
- 5: 计算 $\mathbf{G}_V = \mathbf{G}_{RS} + \mathbf{G}_W$
- 6: 拼装得到 $\mathbf{G}_{\text{sk}} = \begin{pmatrix} \mathbf{G}_U & \mathbf{G}_U \\ \mathbf{G}_V & \mathbf{G}_W \end{pmatrix} \mathbf{D}$, 将其按列分块为 $(\mathbf{A} \parallel \mathbf{B})$, 其中 $\mathbf{A}$ 为左侧 $k \times k$ 的方阵
- 7: 计算 $\mathbf{c}_1 \mathbf{D}^{-1} = (\mathbf{c}_{11} \parallel \mathbf{c}_{12})$
- 8: 计算 $(\mathbf{r}_2, \bar{\mathbf{e}}) = \text{ListDec.RS}(\mathbf{G}_{RS}, \mathbf{c}_{11} - \mathbf{c}_{12}, w)$ 使得 $\mathbf{r}_2 \mathbf{G}_{RS} + \bar{\mathbf{e}} = \mathbf{c}_{11} - \mathbf{c}_{12}$, 其中 $|\bar{\mathbf{e}}| \leq w$
- 9: 计算 $\mathbf{r}_1 = (\mathbf{c}_{12} - \mathbf{r}_2 \mathbf{G}_W)_{I_1} (\mathbf{G}_U)_{I_1}^{-1}$, 其中 $I_1 \subseteq I = [n/2] \setminus \text{supp}(\bar{\mathbf{e}})$ 且集合大小为 k_1
- 10: 计算 $\mathbf{r} = (\mathbf{r}_1 \parallel \mathbf{r}_2) \mathbf{A}$, $\mathbf{e} = \mathbf{c}_1 - \mathbf{r} \mathbf{G}_{\text{pk}}$. 如果 $|\mathbf{e}| \leq w$, 继续; 否则, 返回到第 9 步重新选择一个集合 I_1 计算 $\mathbf{r}_1$

输出: $\mathbf{m} = \mathbf{c}_2 \oplus \mathcal{H}(\mathbf{r}, \mathbf{e})$

正确性: $\mathbf{G}_{\text{pk}}$ 由 $\mathbf{G}_{\text{sk}}$ 经高斯消元化成, 因此有 $\mathbf{G}_{\text{pk}} = (\mathbf{I}_k \parallel \mathbf{T}) = \mathbf{S} \mathbf{G}_{\text{sk}}$, 其中 $\mathbf{S}$ 为行变换可逆矩阵。令 $\mathbf{G}_{\text{sk}} = (\mathbf{A} \parallel \mathbf{B})$, 其中 $\mathbf{A}$ 为左侧的 $k \times k$ 方阵, $\mathbf{B}$ 为右侧 $k \times (n - k)$ 的矩阵, 则有 $\mathbf{S} \cdot (\mathbf{A} \parallel \mathbf{B}) = (\mathbf{I}_k \parallel \mathbf{T})$, 可知 $\mathbf{A} = \mathbf{S}^{-1}$ 。已知 $\mathbf{r} \mathbf{S} \mathbf{G}_{\text{sk}} + \mathbf{e} = \mathbf{c}_1$, 令 $\mathbf{r} \mathbf{S} = (\mathbf{r}_1 \parallel \mathbf{r}_2)$, $\mathbf{e} \mathbf{D}^{-1} = (\mathbf{e}_1 \parallel \mathbf{e}_2)$, $\mathbf{c}_1 \mathbf{D}^{-1} = (\mathbf{c}_{11} \parallel \mathbf{c}_{12})$, 则根据私钥矩阵 $\mathbf{G}_{\text{sk}}$ 的定义, 我们有

$$\mathbf{r}_1 \mathbf{G}_U + \mathbf{r}_2 (\mathbf{G}_{RS} + \mathbf{G}_W) + \mathbf{e}_1 = \mathbf{c}_{11}, \quad \mathbf{r}_1 \mathbf{G}_U + \mathbf{r}_2 \mathbf{G}_W + \mathbf{e}_2 = \mathbf{c}_{12}. \quad (2)$$

以上两个等式相减可得

$$\mathbf{r}_2 \mathbf{G}_{RS} + \mathbf{e}_1 - \mathbf{e}_2 = \mathbf{c}_{11} - \mathbf{c}_{12},$$

这里的向量 $\mathbf{r}_2$ 和 $\mathbf{e}_1 - \mathbf{e}_2$ 就是解密算法第 8 步计算得到的, 其中 $\bar{\mathbf{e}} = \mathbf{e}_1 - \mathbf{e}_2$ 。在求得了向量 $\mathbf{r}_2$ 和 $\bar{\mathbf{e}}$ 之后, 再利用 (2) 中右边的等式, 可得

$$\mathbf{r}_1 \mathbf{G}_U + \mathbf{e}_2 = \mathbf{c}_{12} - \mathbf{r}_2 \mathbf{G}_W.$$

已知 $(\mathbf{e}_1 \parallel \mathbf{e}_2) = \mathbf{e} \mathbf{D}^{-1}$, 令 $I_2 = \text{supp}(\mathbf{e}_2)$ 。如果对于任意的 $i \in I_2$, 有 $\mathbf{e}_2(i) \neq \mathbf{e}_1(i)$, 则包含关系 $\text{supp}(\mathbf{e}_2) \subseteq \text{supp}(\bar{\mathbf{e}})$ 一定成立。利用关系 $\text{supp}(\mathbf{e}_2) \subseteq \text{supp}(\bar{\mathbf{e}})$, 可知向量 $\mathbf{e}_2$ 在集合 $I = [n/2] \setminus \text{supp}(\bar{\mathbf{e}})$ 中所索引的坐标一定没有发生错误, 即对任意的 $i \in I$, 有 $\mathbf{e}_2(i) = 0$ 。所以只要从集合 $I = [n/2] \setminus \text{supp}(\bar{\mathbf{e}})$ 中任意找一个大小为 k_1 的子集 I_1 , 都有

$$\mathbf{r}_1 (\mathbf{G}_U)_{I_1} + (\mathbf{e}_2)_{I_1} = (\mathbf{c}_{12} - \mathbf{r}_2 \mathbf{G}_W)_{I_1},$$

即

$$\mathbf{r}_1 = (\mathbf{c}_{12} - \mathbf{r}_2 \mathbf{G}_W)_{I_1} (\mathbf{G}_U)_{I_1}^{-1}.$$

在得到了向量 $\mathbf{r}_1$ 和 $\mathbf{r}_2$ 后, 那么向量 $\mathbf{r}$ 和 $\mathbf{e}$ 就可以被轻松求得, 从而有

$$\text{Dec}(\text{sk}, \text{Enc}(\text{pk}, \mathbf{m})) = \mathbf{m}. \quad (3)$$

如果存在 $i \in I_2$, 有 $\mathbf{e}_2(i) = \mathbf{e}_1(i)$, 则包含关系 $\text{supp}(\mathbf{e}_2) \subseteq \text{supp}(\bar{\mathbf{e}})$ 不成立, 因此解密算法第 9 步从集合 $I = [n/2] \setminus \text{supp}(\bar{\mathbf{e}})$ 中选择的子集 I_1 有一定的概率包含向量 $\mathbf{e}_2$ 的非零分量的索引, 即 $I_1 \cap \text{supp}(\mathbf{e}_2) \neq \emptyset$, 那么第 10 步求得的向量 $\mathbf{e}$ 将不满足条件 $\text{wt}(\mathbf{e}) = w$ 。此时, 就会重新返回解密算法的第 9 步, 选择新的集合 $I_1 \subseteq I = [n/2] \setminus \text{supp}(\bar{\mathbf{e}})$ 开始重新计算, 直至 $I_1 \cap \text{supp}(\mathbf{e}_2) = \emptyset$, 那么就有

$$\mathbf{r}_1(\mathbf{G}_U)_{I_1} + (\mathbf{e}_2)_{I_1} = (\mathbf{c}_{12} - \mathbf{r}_2 \mathbf{G}_W)_{I_1}, \quad (\mathbf{e}_2)_{i \in I_1}(i) = 0,$$

即

$$\mathbf{r}_1 = (\mathbf{c}_{12} - \mathbf{r}_2 \mathbf{G}_W)_{I_1} (\mathbf{G}_U)_{I_1}^{-1}.$$

在得到了向量 $\mathbf{r}_1$ 和 $\mathbf{r}_2$ 后, 那么向量 $\mathbf{r}$ 和 $\mathbf{e}$ 就可以被正确求出, 从而有

$$\text{Dec}(\text{sk}, \text{Enc}(\text{pk}, \mathbf{m})) = \mathbf{m}. \quad (4)$$

在 UVW 的解密算法中需要判断求得的错误向量 $\mathbf{e}$ 是否满足条件 $\text{wt}(\mathbf{e}) = w$ 。这是由于在 UVW 方案中选择的集合 I_1 有一定的概率包含向量 $\mathbf{e}_2$ 的非零分量的索引, 即 $I_1 \cap \text{supp}(\mathbf{e}_2) \neq \emptyset$, 从而导致解密出错。解密出错概率我们将在 2.2 节具体分析。

1.2.2 UVW 密钥封装机制

本节我们将 Fujisaki-Okamoto 转换 [4] 应用到 UVW 公钥加密方案 UVW.PKE 得到相应的 UVW 密钥封装机制 UVW.KEM。该密钥封装机制的简略描述如下:

假设 Alice 和 Bob 希望共享一个会话密钥, Bob 首先公开自己的密钥 $\text{pk} = \mathbf{T}$, 并保存相应的私钥 $\text{sk} = (\text{seed}, i_1, i_2, i_3)$ 。令 $\mathcal{H}_1, \mathcal{H}_2, \mathcal{H}_3$ 和 $\mathcal{H}_4$ 是四个安全的哈希函数。用 $\text{UVW.PKE.Enc}(\text{pk}, \mathbf{m}; \sigma)$ 表示加密算法 UVW.PKE.Enc(pk, m) 以 σ 为随机种子, 这是一个确定性的加密算法。

算法 6 UVW.KEM.Setup(1^λ): 初始化算法

输入: 安全参数 λ

- 1: 根据安全参数选择相应的系统参数 $\text{params} = (q, n, k, k_1, k_2, w, m)$, 其中 $k = k_1 + k_2$
 - 2: 选择四个安全的哈希函数 $\mathcal{H}_1 : \{0, 1\}^* \rightarrow \mathbb{F}_q^k \times \mathbb{F}_q^n$, $\mathcal{H}_2 : \{0, 1\}^* \rightarrow \mathbb{F}_2^m$, $\mathcal{H}_3 : \{0, 1\}^* \rightarrow \mathbb{F}_2^m$, $\mathcal{H}_4 : \{0, 1\}^* \rightarrow \mathbb{F}_2^{512}$
 - 3: 定义有限域 $\mathbb{F}_q$, 之后的所有运算都在有限域 $\mathbb{F}_q$ 上进行
-

算法 7 UVW.KEM.KeyGen(params): 密钥生成算法

输入: 系统参数 params

- 1: 调用 UVW.PKE 的密钥生成算法 UVW.PKE.KeyGen(params) 生成私钥 sk 和公钥 pk

输出: 私钥 $\text{sk} = (\text{seed}, i_1, i_2, i_3)$, 公钥 $\text{pk} = \mathbf{T}$

算法 8 UVW.KEM.Encap(pk): 密钥封装算法**输入:** 公钥 pk

- 1: 随机选择向量 $\mathbf{m} \in \mathbb{F}_2^m$, 计算 $(\mathbf{r}, \mathbf{e}) = \mathcal{H}_1(\mathbf{m})$ 和 $\mathbf{d} = \mathcal{H}_2(\mathbf{m})$
- 2: 计算 $(\mathbf{c}_1, \mathbf{c}_2) = \text{UVW.PKE.Enc}(\text{pk}, \mathbf{m}, \mathbf{r}, \mathbf{e})$, 其中 $\mathbf{c}_1 = \mathbf{r}(\mathbf{I}_k \parallel \mathbf{T}) + \mathbf{e}$, $\mathbf{c}_2 = \mathbf{m} \oplus \mathcal{H}_3(\mathbf{r}, \mathbf{e})$
- 3: 设置会话密钥 $K = \mathcal{H}_4(\mathbf{m}, \mathbf{c}_1, \mathbf{c}_2)$

输出: $(\mathbf{c}_1, \mathbf{c}_2, \mathbf{d})$ **算法 9** UVW.KEM.Decap(sk, $(\mathbf{c}_1, \mathbf{c}_2, \mathbf{d})$): 密钥解封装算法**输入:** 私钥 sk, 密文 $(\mathbf{c}_1, \mathbf{c}_2, \mathbf{d})$

- 1: 调用 UVW.PKE.Dec(sk, $(\mathbf{c}_1, \mathbf{c}_2)$) 输出 $\mathbf{m}'$
- 2: 计算 $(\mathbf{r}', \mathbf{e}') = \mathcal{H}_1(\mathbf{m}')$ 和 $\mathbf{d}' = \mathcal{H}_2(\mathbf{m}')$
- 3: 计算 $(\mathbf{c}'_1, \mathbf{c}'_2) = \text{UVW.PKE.Enc}(\text{pk}, \mathbf{m}', \mathbf{r}', \mathbf{e}')$
- 4: 如果 $(\mathbf{c}_1, \mathbf{c}_2, \mathbf{d}) = (\mathbf{c}'_1, \mathbf{c}'_2, \mathbf{d}')$, 继续; 否则, 中止

输出: $K = \mathcal{H}_4(\mathbf{m}, \mathbf{c}_1, \mathbf{c}_2)$

1.3 算法实例与参数

根据现有译码攻击和码区分攻击的最优攻击算法, 我们为 UVW 密钥封装机制选择了三组参数集, 它们分别对应 128 bit、256 bit、512 bit 经典安全强度等级, 其量子安全强度分别不低于 80 bit、128 bit、256 bit。

表 1: UVW 密钥封装机制推荐参数集

方案	$(q, n, k, k_1, k_2, w, m)$	公钥 pk
UVW128	(433, 860, 430, 215, 215, 116, 256)	208013 Bytes
UVW256	(857, 1708, 854, 427, 427, 232, 256)	911645 Bytes
UVW512	(1709, 3412, 1706, 853, 853, 463, 512)	4001850 Bytes

方案中使用的哈希函数由基于 SM3 的可扩展输出函数算法来实例化。

2 设计原理

2.1 数学困难问题

一般译码 (General Decoding) 问题是基于编码的密码方案中使用最广泛的问题之一, 我们的 UVW 方案的安全性也基于该问题。校验子译码问题 (Syndrome Decoding) 是一般译码问题的对偶问题, 在密码分析中, 这两个问题是计算等价的。

问题 1 (一般译码 (General Decoding) 问题 (GD)). 给定有限域 $\mathbb{F}_q$ 上的一个 $[n, k]$ 随机线性码 $\mathcal{C}$ 的生成矩阵 $\mathbf{G} \in \mathbb{F}_q^{k \times n}$, 一个向量 $\mathbf{r} \in \mathbb{F}_q^n$ 以及一个正整数 w 。目的是找到一个向量 $\mathbf{m} \in \mathbb{F}_q^k$ 使得 $wt(\mathbf{m}\mathbf{G} - \mathbf{r}) \leq w$ 。

问题 2 (校验子译码 (Syndrome Decoding) 问题 (SD)). 给定有限域 $\mathbb{F}_q$ 上的一个 $[n, k]$ 随机线性码 $\mathcal{C}$ 的校验矩阵 $\mathbf{H} \in \mathbb{F}_q^{(n-k) \times n}$, 一个向量 $\mathbf{s} \in \mathbb{F}_q^{n-k}$ 以及一个正整数 w 。目的是找到一个向量 $\mathbf{e} \in \mathbb{F}_q^n$ 使得 $\mathbf{H}\mathbf{e}^T = \mathbf{s}^T$ 且 $wt(\mathbf{e}) \leq w$ 。

一般译码问题的困难性是关于随机码的, 而我们是利用广义 $(U + V, U + W)$ 码来构造密码方案, 因此我们也必须考虑广义 $(U + V, U + W)$ 码区分问题 (CD) 的困难性。

令 $\mathcal{G}_{\text{pk}}$ 是 UVW.PKE 方案中所有可能生成的公钥矩阵构成的集合。 $\mathcal{G}_{\text{pk}}$ 区分器 $\mathcal{D}$ 是一个概率多项式时间算法。我们如下定义它的优势:

$$\text{Adv}^{CD}(\mathcal{D}) = \left| \mathbb{P}[\mathbf{G} \xleftarrow{\$} \mathcal{G}_{\text{pk}} : \mathcal{D}(\mathbf{G}) = 1] - \mathbb{P}[\mathbf{G} \xleftarrow{\$} \mathbb{F}_q^{k \times n} : \mathcal{D}(\mathbf{G}) = 1] \right|. \quad (5)$$

如果对任意区分器 $\mathcal{D}$ 来说, 上述优势是可忽略的, 则 UVW.PKE 方案的公钥矩阵与随机矩阵是不可区分的, 也即广义 $(U + V, U + W)$ 码与随机线性码是不可区分的。

定义 8. 令 $\mathcal{G}_{\text{pk}}$ 是 UVW.PKE 方案中所有可能生成的公钥矩阵构成的集合。如果对于任意一个运行时间至多为 τ 的 $\mathcal{G}_{\text{pk}}$ 区分器 $\mathcal{D}$, 有

$$\text{Adv}^{CD}(\mathcal{D}) \leq \epsilon,$$

则广义 $(U + V, U + W)$ 码区分问题是 (τ, ϵ) -安全的。

2.2 失败概率分析及影响

在 UVW 公钥加密方案的解密算法 (第 9 步) 中, 解密者需要从推断出的无错坐标索引集 $I = [n/2] \setminus \text{supp}(\bar{e})$ 中, 随机抽取一个大小为 k_1 的子集 I_1 。由于这种抽取具有随机性, 如果选出的子集 I_1 恰好包含了错误向量分量 e_2 的非零索引 (即 $I_1 \cap \text{supp}(e_2) \neq \emptyset$), 解密算法将无法正确求解出满足重量约束 $wt(e) = w$ 的错误向量, 从而引发单次解密失败。为了准确评估方案的可靠性, 必须量化这种因随机采样导致的失败情况。下面的定理 2 给出了单次随机选取集合 I_1 时, 能够完美避开 e_2 支撑集 (即 $I_1 \cap \text{supp}(e_2) = \emptyset$, 单次解密成功) 的概率。

定理 2. 令 I_1 是 UVW 方案中解密算法第 9 步随机选择的集合, 则 $I_1 \cap \text{supp}(e_2) = \emptyset$ 的概率为

$$\mathbb{P}(I_1 \cap \text{supp}(e_2) = \emptyset) = \frac{1}{\binom{n}{w}(q-1)^w} \left(\sum_{t=0}^{\frac{w}{2}} \sum_{i=0}^{w-2t} \binom{\frac{n}{2}}{t} \binom{\frac{n-2t}{2}}{i} \binom{\frac{n-2t-2i}{2}}{\frac{w-2t-i}{2}} \right. \\ \left. 2^i (q-2)^{\frac{w-2t-i}{2}} (q-1)^{\frac{w+i}{2}} \binom{\frac{\frac{n-w-i}{2}}{k_1}}{\frac{n-w+2t-i}{2}} \right).$$

证明. 令 $\mathbf{e} = (\mathbf{e}_1 || \mathbf{e}_2)$ 是从集合 $\mathcal{E}_{q,n}^w$ 中均匀随机选择的一个重量为 w 的向量, 其中 $\mathbf{e}_1, \mathbf{e}_2 \in \mathbb{F}_q^{n/2}$ 。定义如下三个中间重量:

$$\begin{aligned} |\mathbf{e}|^1 &= |\{i \in [n/2] \mid \mathbf{e}_j(i) = 0, \mathbf{e}_{3-j}(i) \neq 0, j = 1, 2\}|, \\ |\mathbf{e}|^2 &= |\{i \in [n/2] \mid \mathbf{e}_1(i) \neq 0, \mathbf{e}_2(i) \neq 0, \mathbf{e}_1(i) \neq \mathbf{e}_2(i)\}|, \\ |\mathbf{e}|^3 &= |\{i \in [n/2] \mid \mathbf{e}_1(i) \neq 0, \mathbf{e}_2(i) \neq 0, \mathbf{e}_1(i) = \mathbf{e}_2(i)\}|, \end{aligned}$$

则以下两个等式成立

$$|\mathbf{e}| = |\mathbf{e}|^1 + 2|\mathbf{e}|^2 + 2|\mathbf{e}|^3, \quad |\mathbf{e}_1 - \mathbf{e}_2| = |\mathbf{e}|^1 + |\mathbf{e}|^2.$$

我们发现, 当 $|\mathbf{e}|^3 = 0$ 时, $\text{supp}(\mathbf{e}_2) \subseteq \text{supp}(\mathbf{e}_1 - \mathbf{e}_2)$, 那么事件 $I_1 \cap \text{supp}(\mathbf{e}_2) = \emptyset$ 总是发生; 而当 $|\mathbf{e}|^3 \neq 0$ 时, 事件 $I_1 \cap \text{supp}(\mathbf{e}_2) = \emptyset$ 和 $I_1 \cap \text{supp}(\mathbf{e}_2) \neq \emptyset$ 都有可能发生。

根据 $|\mathbf{e}|^1$ 、 $|\mathbf{e}|^2$ 和 $|\mathbf{e}|^3$ 的定义, 我们有

$$\begin{aligned} \mathbb{P}(I_1 \cap \text{supp}(\mathbf{e}_2) = \emptyset) &= \sum_{t=0}^{\frac{w}{2}} \sum_{i=0}^{w-2t} \mathbb{P}(|\mathbf{e}|^3 = t, |\mathbf{e}|^1 = i, |\mathbf{e}|^2 = \frac{w-2t-i}{2}) \frac{\binom{\frac{n}{2}-i-\frac{w-2t-i}{2}-t}{k_1}}{\binom{\frac{n}{2}-i-\frac{w-2t-i}{2}}{k_1}} \\ &= \sum_{t=0}^{\frac{w}{2}} \sum_{i=0}^{w-2t} \mathbb{P}(|\mathbf{e}|^3 = t, |\mathbf{e}|^1 = i, |\mathbf{e}|^2 = \frac{w-2t-i}{2}) \frac{\binom{\frac{n-w-i}{2}}{k_1}}{\binom{\frac{n-w+2t-i}{2}}{k_1}} \\ &= \sum_{t=0}^{\frac{w}{2}} \sum_{i=0}^{w-2t} \left(\frac{1}{\binom{n}{w}(q-1)^w} \binom{n/2}{t} (q-1)^t \binom{n/2-t}{i} (2q-2)^i \right. \\ &\quad \left. \binom{n/2-t-i}{\frac{w-2t-i}{2}} ((q-2)(q-1))^{\frac{w-2t-i}{2}} \right) \frac{\binom{\frac{n-w-i}{2}}{k_1}}{\binom{\frac{n-w+2t-i}{2}}{k_1}} \\ &= \frac{1}{\binom{n}{w}(q-1)^w} \left(\sum_{t=0}^{\frac{w}{2}} \sum_{i=0}^{w-2t} \binom{n}{t} \binom{\frac{n-2t}{2}}{i} \binom{\frac{n-2t-2i}{2}}{\frac{w-2t-i}{2}} \right. \\ &\quad \left. 2^i (q-2)^{\frac{w-2t-i}{2}} (q-1)^{\frac{w+i}{2}} \frac{\binom{\frac{n-w-i}{2}}{k_1}}{\binom{\frac{n-w+2t-i}{2}}{k_1}} \right). \end{aligned}$$

这就完成了定理 2 的证明。 $\square$

这里我们想要强调, 对于我们在第 1.3 节中选择的参数, $I_1 \cap \text{supp}(\mathbf{e}_2) = \emptyset$ 的概率接近 1。具体来说, 对于 UVW128、UVW256 和 UVW512 的参数, 单次解密成功概率分别为 0.988123、0.987827 和 0.987803。

2.3 参数选择依据

为兼顾方案的安全性与实现性能, 本方案在系统参数 $params = (q, n, k, k_1, k_2, w, m)$ 的选取上, 综合考虑了以下两方面:

可靠性 如在 2.2 节所述, 为确保在解密算法中子集 I_1 的选取不包含错误向量的非零分量 (即确保 $I_1 \cap \text{supp}(\mathbf{e}_2) = \emptyset$), 参数 q 、 n 、 k_1 与 w 的选取必须保证极高的单次求解成功率。经参数选优, 我们选取的参数集确保了解密成功率趋近于 1, 从而保障了方案的可靠性。

安全性 选取的系统参数必须足够抵抗现有最优的密码分析方法。我们主要考量了针对随机线性码的一般译码攻击 (ISD 算法、量子 ISD 算法) 以及针对广义 $(U+V, U+W)$ 码结构的区分攻击, 详见本文第 3.2 节 “具体安全性”。

3 安全性声明与分析

3.1 理论安全性

选择明文攻击下的不可区分安全 (IND-CPA) 和选择密文攻击下的不可区分安全 (IND-CCA) 是公钥加密方案中最常用的两个安全模型。在 IND-CPA 安全模型中, 敌手可以查询任意消息对应的密文, 其目标是在给定密文的情况下, 分辨出它是由两条候选明文中的哪一条加密得到的。

设 $\Pi = (\text{KeyGen}, \text{Encrypt}, \text{Decrypt})$ 是一个公钥加密方案, 下面给出其 IND-CPA 安全的形式化定义:

定义 9 (IND-CPA 安全). 令 $\Pi = (\text{KeyGen}, \text{Encrypt}, \text{Decrypt})$ 是一个公钥加密方案, λ 是安全参数。如果对任意的概率多项式时间敌手 $\mathcal{A}$, 下面的实验 $\text{Exp}_{\mathcal{A}, \Pi}^{\text{IND-CPA}}(1^\lambda)$ 返回值为 1 的概率可忽略的接近 $1/2$, 即

$$|\mathbb{P}[\text{Exp}_{\mathcal{A}, \Pi}^{\text{IND-CPA}}(1^\lambda) = 1] - \frac{1}{2}| \leq \text{negl}(\lambda),$$

则公钥加密方案 $\Pi = (\text{KeyGen}, \text{Encrypt}, \text{Decrypt})$ 满足 IND-CPA 安全。

$\text{Exp}_{\mathcal{A}, \Pi}^{\text{IND-CPA}}(1^\lambda)$:

- 1) $(pk, sk) \leftarrow \text{KeyGen}(1^\lambda)$;
- 2) $(\mathbf{m}_0, \mathbf{m}_1) \leftarrow \mathcal{A}(pk, 1^\lambda)$ (即 $\mathcal{A}$ 选择两个安全参数 λ 多项式大小的消息, 另外, 在公钥加密方案中, 敌手拥有公钥, 等价于敌手可以访问加密预言机, 这里省略加密询问步骤);
- 3) $b \xleftarrow{\$} \{0, 1\}$, 挑战者 $\mathcal{C}$ 计算 $\mathbf{c} \leftarrow \text{Encrypt}(pk, \mathbf{m}_b)$, 并将 $\mathbf{c}$ 发送给敌手 $\mathcal{A}$;
- 4) $b' \leftarrow \mathcal{A}(pk, 1^\lambda, \mathbf{c})$;
- 5) 如果 $b' = b$ 则返回 1, 否则返回 0。

相比于 IND-CPA 安全, 更强的安全模型是 IND-CCA 安全, 其中敌手被允许访问解密预言机。IND-CCA 安全模型进一步细分为 IND-CCA1 和 IND-CCA2。在 IND-CCA1 模型中, 敌手可以在选择挑战消息对 $(\mathbf{m}_0, \mathbf{m}_1)$ 之前, 任意查询解密预言机。在 IND-CCA2 模型中, 敌手不仅可以在选择消息对 $(\mathbf{m}_0, \mathbf{m}_1)$ 之前访问解密预言机, 还可以在挑战者返回密文 $\mathbf{c}$ 之后继续查询解密预言机 (但不能询问密文 $\mathbf{c}$ 的解密结果)。对于一个已满足 IND-CPA 安全性的公钥加密方案, 可以通过通用转换 (如 Naor-Yung 转换 [5] 或 Fujisaki-Okamoto 转换 [4]) 提升安全性, 从而实现 IND-CCA 安全。

由于 UVW.KEM 密钥封装方案是通过将 Fujisaki-Okamoto 转化应用到 UVW.PKE 公钥加密方案得来的, 因此, 只要证明了 UVW.PKE 公钥加密方案是 IND-CPA 安全的, 那么 UVW.KEM 密钥封装方案就是 IND-CCA 安全的。

这里我们给出 UVW.PKE 公钥加密方案的安全性证明。当译码问题和码区分问题困难时, 我们证明了公钥加密方案在随机预言机模型中是 IND-CPA 安全的。

定理 3. 令 t_{enc} 为执行一次加密操作所需的时间, t_H 为响应一次随机预言机查询所需的时间, q_H 为敌手发起哈希查询的最大次数。假设 $\mathcal{G}_{pk}$ 码区分问题 (CD) 是 $(\tau_{CD}, \epsilon_{CD})$ -安全的, 且一般译码问题 (GD) 是 $(\tau_{GD}, \epsilon_{GD})$ -安全的。那么, UVW.PKE 公钥加密方案在随机预言机模型中是 (τ', ϵ') -IND-CPA 安全的, 其中: $\epsilon' = 2(\epsilon_{CD} + \epsilon_{GD})$, $\tau' \approx \min(\tau_{CD}, \tau_{GD}) - q_H \cdot t_H - t_{enc}$ 。

证明. 我们将通过构造一系列游戏来证明加密方案是满足 IND-CPA 安全的, 其中的每一个游戏之间都是计算不可区分的, 最终得出加密消息 $\mathbf{m}_0$ 和加密消息 $\mathbf{m}_1$ 生成的两个密文之间是不可区分的。如果存在敌手 $\mathcal{A}$ 能够区分这一系列游戏中相邻的两个游戏, 那么我们就利用敌手 $\mathcal{A}$ 的能力构造算法 $\mathcal{B}$ 将公钥矩阵与随机矩阵区分开来或解决一般译码问题。

设 **Game 0** (图 1) 是公钥加密方案加密消息 $\mathbf{m}_0$ 的真实的 IND-CPA 游戏。我们用 $\mathbb{P}(S_i)$ 表示敌手 $\mathcal{A}$ 在游戏 **Game** i 中输出 $b' = 0$ 的概率, 则我们有

$$\mathbb{P}(S_0) = \mathbb{P}[b' \leftarrow \mathcal{A}(\text{pk}, \lambda, (\mathbf{c}_1, \mathbf{c}_2)), b' = 0]. \quad (6)$$

```

1  $params \leftarrow \text{Setup}(1^\lambda)$ 
2  $(\text{pk}, \text{sk}) \leftarrow \text{KeyGen}(params)$ 
3  $(\mathbf{m}_0, \mathbf{m}_1) \leftarrow \mathcal{A}(\text{pk}, \lambda)$ 
4  $(\mathbf{c}_1, \mathbf{c}_2) = (\mathbf{r}\mathbf{G}_{\text{pk}} + \mathbf{e}, \mathbf{m}_0 \oplus \text{Hash}(\mathbf{r}, \mathbf{e})) \leftarrow \text{Encrypt}(\text{pk}, \mathbf{m}_0)$ 
5  $b' \leftarrow \mathcal{A}(\text{pk}, \lambda, (\mathbf{c}_1, \mathbf{c}_2))$ 

```

图 1: **Game 0**: 加密消息 $\mathbf{m}_0$ 的真实的 IND-CPA 游戏

在 **Game 0** 中, 向量 $\mathbf{r}$ 是从向量空间 $\mathbb{F}_q^k$ 中随机选择的向量, 而向量 $\mathbf{e} \in \mathbb{F}_q^n$ 是从向量空间 $\mathbb{F}_q^n$ 中随机选择的一个长为 n , 重量为 w 且非零分量都不相同的向量。

```

1  $params \leftarrow \text{Setup}(1^\lambda)$ 
2  $(\text{pk}, \text{sk}) \leftarrow \text{KeyGen}(params)$ 
3  $\text{pk} \leftarrow \mathbf{G}$ 
4  $(\mathbf{m}_0, \mathbf{m}_1) \leftarrow \mathcal{A}(\text{pk}, \lambda)$ 
5  $(\mathbf{c}_1, \mathbf{c}_2) = (\mathbf{r}\mathbf{G} + \mathbf{e}, \mathbf{m}_0 \oplus \text{Hash}(\mathbf{r}, \mathbf{e})) \leftarrow \text{Encrypt}(\text{pk}, \mathbf{m}_0)$ 
6  $b' \leftarrow \mathcal{A}(\text{pk}, \lambda, (\mathbf{c}_1, \mathbf{c}_2))$ 

```

图 2: **Game 1**: 随机矩阵 $\mathbf{G}$ 代替公钥矩阵 $\mathbf{G}_{\text{pk}}$

Game 1. 在这个游戏中, 挑战者用随机矩阵 $\mathbf{G}$ 代替公钥矩阵 $\mathbf{G}_{\text{pk}}$ (图 2)。为了建立 **Game 0** 和 **Game 1** 之间的联系, 我们构造一个区分器 $\mathcal{D}$ 。如果公钥 pk 是由密钥生成算法 KeyGen 生成的, 则区分器 $\mathcal{D}$ 在进行 **Game 0**, 此时, 我们有

$$\mathbb{P}[\mathbf{G}_{\text{pk}} \leftarrow \text{KeyGen}(params) : \mathcal{D}(\mathbf{G}_{\text{pk}}) = 1] = \mathbb{P}(S_0).$$

如果公钥 pk 中的生成矩阵是随机矩阵, 则区分器 $\mathcal{D}$ 在进行 **Game 1**, 此时, 我们有

$$\mathbb{P}[\mathbf{G} \xleftarrow{\$} \mathbb{F}_q^{k \times n} : \mathcal{D}(\mathbf{G}) = 1] = \mathbb{P}(S_1).$$

因此, 区分器 $\mathcal{D}$ 区分公钥矩阵 $\mathbf{G}_{\text{pk}}$ 和随机矩阵 $\mathbf{G}$ 的优势如下

$$\text{Adv}^{CD}(\mathcal{D}) = |\mathbb{P}(S_0) - \mathbb{P}(S_1)|,$$

又由于 $\mathcal{G}_{pk}$ 码区分问题是 $(\tau_{CD}, \epsilon_{CD})$ -安全的, 我们有

$$|\mathbb{P}(S_0) - \mathbb{P}(S_1)| \leq \epsilon_{CD}. \quad (7)$$

```

1  $params \leftarrow \text{Setup}(1^\lambda)$ 
2  $(pk, sk) \leftarrow \text{KeyGen}(params)$ 
3  $(m_0, m_1) \leftarrow \mathcal{A}(pk, \lambda)$ 
4  $(c_1, c_2) = (rG + e, m_0 \oplus s) \leftarrow \text{Encrypt}(pk, m_0)$ 
5  $b' \leftarrow \mathcal{A}(pk, \lambda, (c_1, c_2))$ 

```

图 3: **Game 2**: 随机比特串 s 代替 $\text{Hash}(r, e)$

Game 2. 在这个游戏中, 挑战者用随机比特串 s 代替 $\text{Hash}(r, e)$ (图 3)。我们首先分析 **Game 2** 中 $\mathbb{P}(S_2)$ 的取值。从 **Game 2** 可以看出, 密文 (c_1, c_2) 中的 c_1 和 c_2 的取值是相互独立的。由于 r , G 和 e 的选择和消息 m_0 无关, 所以 c_1 和消息 m_0 是相互独立的。已知密文 $c_2 = m_0 \oplus s$, 其中 s 是随机比特串, 因此 c_2 和消息 m_0 也是相互独立的。综上所述, 密文 (c_1, c_2) 和消息 m_0 是相互独立的, 也就是说敌手只能随机猜测输出 b' , 即

$$\mathbb{P}(S_2) = \frac{1}{2}. \quad (8)$$

为了分析 **Game 1** 和 **Game 2** 的不可区分性, 我们需要监控敌手对随机预言机 Hash 的查询。令 Query 表示敌手 $\mathcal{A}$ 向随机预言机 Hash 查询了用于生成挑战密文的向量对 (r, e) 的事件。

1. 若 Query 事件不发生: 在随机预言机模型下, 对于敌手未查询过的输入, 预言机 $\text{Hash}(r, e)$ 的输出与随机比特串 s 在统计上是不可区分的。此时 **Game 1** 和 **Game 2** 对敌手而言完全相同。
2. 若 Query 事件发生: 这意味敌手成功找到了 (r, e) 。我们将 Query 事件分为两种情况:
 - 情况 A (挑战前查询): 敌手在看到挑战密文之前就查询了 (r, e) 。由于 r, e 是在挑战阶段从巨大的空间中均匀随机选取的, 且独立于敌手之前的视图, 该情况发生的概率 $\leq q_H/|\mathcal{S}|$, 是可以忽略的, $|\mathcal{S}|$ 表示哈希的输入值 (r, e) 的样本空间总大小。
 - 情况 B (挑战后查询): 敌手在收到 $c_1 = rG + e$ 后查询了 (r, e) 。此时 G 是随机矩阵, 这意味着敌手在随机线性码上成功解出了错误向量 e 和向量 r 。这等价于求解一般译码问题。

因此, **Game 1** 和 **Game 2** 的区别被 Query 事件的概率所界定, 且主要取决于情况 B (即一般译码问题的困难性):

$$|\mathbb{P}(S_1) - \mathbb{P}(S_2)| \leq \mathbb{P}(\text{Query}) \leq \epsilon_{GD} \quad (9)$$

根据式子 (7), (8) 和 (9), 我们有

$$|\mathbb{P}(S_0) - \frac{1}{2}| \leq \epsilon_{GD} + \epsilon_{CD}. \quad (10)$$

接下来, 从本游戏开始, 挑战者对另一个消息 $\mathbf{m}_1$ 进行加密。设 **Game 3** (图 4) 是公钥加密方案加密消息 $\mathbf{m}_1$ 的真实的 IND-CPA 游戏。与之前类似, 我们用 $\mathbb{P}(S_i)$ 表示敌手 $\mathcal{A}$ 在游戏 **Game i** 中输出 $b' = 0$ 的概率, 则我们有

$$\mathbb{P}(S_3) = \mathbb{P}[b' \leftarrow \mathcal{A}(\text{pk}, \lambda, (\mathbf{c}_1, \mathbf{c}_2)), b' = 0]. \quad (11)$$

```

1  $params \leftarrow \text{Setup}(1^\lambda)$ 
2  $(\text{pk}, \text{sk}) \leftarrow \text{KeyGen}(params)$ 
3  $(\mathbf{m}_0, \mathbf{m}_1) \leftarrow \mathcal{A}(\text{pk}, \lambda)$ 
4  $(\mathbf{c}_1, \mathbf{c}_2) = (\mathbf{r}\mathbf{G}_{\text{pk}} + \mathbf{e}, \mathbf{m}_1 \oplus \text{Hash}(\mathbf{r}, \mathbf{e})) \leftarrow \text{Encrypt}(\text{pk}, \mathbf{m}_1)$ 
5  $b' \leftarrow \mathcal{A}(\text{pk}, \lambda, (\mathbf{c}_1, \mathbf{c}_2))$ 

```

图 4: **Game 3**: 加密消息 $\mathbf{m}_1$ 的真实的 IND-CPA 游戏

Game 3 和 **Game 0** 之间的唯一区别是将消息 $\mathbf{m}_0$ 换成了消息 $\mathbf{m}_1$ 。

Game 4. 在这个游戏中, 挑战者用随机矩阵 $\mathbf{G}$ 代替公钥矩阵 $\mathbf{G}_{\text{pk}}$ 。**Game 3** 和 **Game 4** 之间的关系与前面 **Game 0** 和 **Game 1** 之间的关系相同, 即有

$$|\mathbb{P}(S_3) - \mathbb{P}(S_4)| \leq \epsilon_{CD}. \quad (12)$$

Game 5. 在这个游戏中, 挑战者用随机比特串 $\mathbf{s}$ 代替 $\text{Hash}(\mathbf{r}, \mathbf{e})$ 。类似地, **Game 5** 和 **Game 2** 相同, 敌手也只能随机猜测输出 b' , 即

$$\mathbb{P}(S_5) = \frac{1}{2}. \quad (13)$$

此外, **Game 4** 和 **Game 5** 之间的关系与前面 **Game 1** 和 **Game 2** 之间的关系也是相同的, 所以有

$$|\mathbb{P}(S_4) - \mathbb{P}(S_5)| \leq \epsilon_{GD}. \quad (14)$$

根据式子 (12), (13) 和 (14), 我们有

$$|\mathbb{P}(S_3) - \frac{1}{2}| \leq \epsilon_{GD} + \epsilon_{CD}. \quad (15)$$

最后, 结合式子 (10) 和 (15), 我们有

$$|\mathbb{P}(S_0) - \mathbb{P}(S_3)| \leq 2(\epsilon_{GD} + \epsilon_{CD}), \quad (16)$$

即

$$\begin{aligned} \text{Adv}_{\mathcal{A}}^{\text{IND-CPA}} &= |\mathbb{P}[b' \leftarrow \mathcal{A}(\text{pk}, \lambda, \text{Encrypt}(\text{pk}, \mathbf{m}_0)), b' = 0] \\ &\quad - \mathbb{P}[b' \leftarrow \mathcal{A}(\text{pk}, \lambda, \text{Encrypt}(\text{pk}, \mathbf{m}_1)), b' = 0]| \leq 2(\epsilon_{GD} + \epsilon_{CD}). \end{aligned} \quad (17)$$

从而

$$\epsilon' = 2(\epsilon_{CD} + \epsilon_{GD}). \quad (18)$$

关于运行时间 τ' ，上述归约过程通过调用敌手 $\mathcal{A}$ 来构建算法 $\mathcal{B}$ 解决 CD 或 GD 问题。算法 $\mathcal{B}$ 需要运行敌手 $\mathcal{A}$ 和模拟加密预言机（耗时 t_{enc} ）并响应最多 q_H 次哈希查询（耗时 $q_H \cdot t_H$ ）。因此，敌手的有效运行时间 τ' 受限于底层困难问题的求解时间减去模拟开销，即：

$$\tau' \approx \min(\tau_{CD}, \tau_{GD}) - (q_H \cdot t_H + t_{enc}). \quad (19)$$

□

3.2 具体安全性

3.2.1 经典安全强度分析

UVW 方案的安全性基于两个困难假设：1) 随机线性码的译码问题（即问题 1）；2) 判断给定的线性码是否是广义 $(U + V, U + W)$ 码。

第一个随机线性码的译码问题早已被证明是 NP 完全问题，当参数选择合适时，即使是量子计算机也需要指数时间求解。第二个困难假设使得公开的公钥矩阵 $\mathbf{G}_{pk} = \mathbf{S}\mathbf{G}_{sk}\mathbf{D}$ 与随机矩阵是不可区分的。从结构来看， $(U + V, U + W)$ 码看起来与随机码非常相似。相同长度和维数的随机码与 $(U + V, U + W)$ 码之间只有非常细微的差异。而且这些差异只发生在小重量和大重量码字之间。当 $\mathbf{v} = \mathbf{w} = \mathbf{0}$ ，码字的形式为 $(\mathbf{u}, \mathbf{u})\mathbf{D}$ ；而当 $\mathbf{u} = \mathbf{0}$ ，码字的形式为 $(\mathbf{v}, \mathbf{w})\mathbf{D}$ 。这两种情况或许都能被用来作为判断给定的线性码是否是置换 $(U + V, U + W)$ 码。因此，第二个困难假设可以归结为最小重量或最大重量码字寻找问题，这本质上与第一个困难假设是一样的。命题 4 给出了有限域 $\mathbb{F}_q$ 上的一个 $[n, k]$ Reed-Solomon 码的码字重量分布。

命题 4. 令 $\mathcal{C}$ 是有限域 $\mathbb{F}_q$ 上的一个 $[n, k]$ Reed-Solomon 码。令 $a_1 = q - 1$ ， $a_i = q^i - 1 - \sum_{j=1}^{i-1} \binom{n-k+i}{n-k+j} a_j$ ($i = 2, \dots, k$)。令 N_i 是码 $\mathcal{C}$ 中重量为 $n - k + i$ 的码字的个数，则

$$N_i = \binom{n}{n-k+i} a_i.$$

证明. 已知 Reed-Solomon 码是极大距离可分码，因此，给定一个 $[n, k]$ Reed-Solomon 码 $\mathcal{C}$ ，则码 $\mathcal{C}$ 中码字的重量范围为 $[n - k + 1; n]$ 。我们知道一个 $[n, k]$ Reed-Solomon 码 $\mathcal{C}$ 的对偶码 $\mathcal{C}^\perp$ 是一个 $[n, n - k]$ Reed-Solomon 码。由对偶码定义可知， $\mathbf{x} \in \mathcal{C}$ 当且仅当

$$\mathbf{x}\mathbf{H}^\top = \mathbf{0}, \quad (20)$$

其中 $\mathbf{H}$ 是码 $\mathcal{C}^\perp$ 的生成矩阵。因此，求

Reed-Solomon 码 $\mathcal{C}$ 的重量分布等价于求线性方程组 (20) 的解的重量分布。

已知 $\mathbf{H}$ 是一个 $[n, n - k]$ Reed-Solomon 码的生成矩阵，根据 Reed-Solomon 码的定义，任取 $\mathbf{H}$ 的 $n - k$ 列，这 $n - k$ 列构成一个范德蒙矩阵，即行列式不等于 0（或者说这 $n - k$ 列线性无关），因此线性方程组 (20) 不存在重量为 $n - k$ 的解。又由于任取 $n - k + 1$ 列，这 $n - k + 1$ 列一定线性相关，必定存在非零解，也可看成求一个线性方程组，其中有 $n - k$ 个方程， $n - k + 1$ 个未知数，具有 1 个自由变量，所以线性方程组 (20) 存在重量为 $n - k + 1$ 的解，且重量为 $n - k + 1$ 的解的个数为

$$\binom{n}{n-k+1} (q - 1).$$

接下来求线性方程组 (20) 有多少个重量为 $n-k+2$ 的解。从 $\mathbf{H}$ 中任取 $n-k+2$ 列组成一个具有 $n-k$ 个方程、 $n-k+2$ 个未知数的线性方程组。该方程组有 2 个自由变量，因此，非零解的个数为

$$q^2 - 1.$$

再除去其中重量为 $n-k+1$ 的解，则一个具有 $n-k$ 个方程、 $n-k+2$ 个未知数的线性方程组中重量为 $n-k+2$ 的解的个数为

$$q^2 - 1 - \binom{n-k+2}{n-k+1}(q-1).$$

因此，线性方程组 (20) 中重量为 $n-k+2$ 的解的个数为

$$\binom{n}{n-k+2} \left(q^2 - 1 - \binom{n-k+2}{n-k+1}(q-1) \right).$$

令

$$a_1 = q - 1, \quad a_2 = q^2 - 1 - \binom{n-k+2}{n-k+1}(q-1).$$

接下来我们求线性方程组 (20) 中重量为 $n-k+3$ 的解的个数。

同理，从 $\mathbf{H}$ 中任取 $n-k+3$ 列组成一个具有 $n-k$ 个方程、 $n-k+3$ 个未知数的线性方程组。该方程组有 3 个自由变量。因此，非零解的个数为

$$q^3 - 1.$$

再除去其中重量为 $n-k+1$ 和 $n-k+2$ 的解，则一个具有 $n-k$ 个方程、 $n-k+3$ 个未知数的线性方程组中重量为 $n-k+3$ 的解的个数为

$$\begin{aligned} & q^3 - 1 - \binom{n-k+3}{n-k+1}(q-1) - \binom{n-k+3}{n-k+2} \left(q^2 - 1 - \binom{n-k+2}{n-k+1}(q-1) \right) \\ &= q^3 - 1 - \sum_{j=1}^2 \binom{n-k+3}{n-k+j} a_j. \end{aligned}$$

令

$$a_3 = q^3 - 1 - \sum_{j=1}^2 \binom{n-k+3}{n-k+j} a_j,$$

则线性方程组 (20) 中重量为 $n-k+3$ 的解的个数为

$$\binom{n}{n-k+3} a_3.$$

从上述分析，我们发现从 $\mathbf{H}$ 中任取 $n-k+i$ 列组成一个具有 $n-k$ 个方程、 $n-k+i$ 个未知数的线性方程组，其重量为 $n-k+i$ 的解的个数等于所有非零解的个数 $q^i - 1$ 除去该方程组中重量分别为 $n-k+1, n-k+2, \dots, n-k+i-1$ 的解的个数，因此，线性方程组 (20) 中重量为 $n-k+i$ 的解的个数为

$$\binom{n}{n-k+i} a_i,$$

其中

$$a_i = q^i - 1 - \sum_{j=1}^{i-1} \binom{n-k+i}{n-k+j} a_j.$$

□

在有了 Reed-Solomon 码的码字重量分布之后, 下面我们给出 Reed-Solomon 码与相同大小的随机线性码之间的码字重量分布的统计距离。

命题 5. 令 C_1 和 C_2 分别是有限域 $\mathbb{F}_q$ 上的一个 $[n, k]$ Reed-Solomon 码和一个 $[n, k]$ 随机线性码。则 C_1 和 C_2 之间的码字重量分布的统计距离为

$$\rho(|C_1|, |C_2|) = \frac{1}{2} \left(\sum_{t=1}^{n-k} \frac{\binom{n}{t}(q-1)^t}{q^n} + \sum_{t=n-k+1}^n \left| \frac{\binom{n}{t}(q-1)^t}{q^n} - \frac{N_{t-n+k}}{q^k} \right| \right).$$

证明. 根据命题 4, 可知 Reed-Solomon 码 C_1 中的码字重量分布分为两个部分:

- 1) 当 $t \in [n-k]$ 时, 码 C_1 中重量为 t 的码字的比例为 0。
- 2) 当 $t \in [n-k+1; n]$ 时, 码 C_1 中重量为 t 的码字的比例为

$$\frac{N_{t-n+k}}{q^k}.$$

下面给出有限域 $\mathbb{F}_q$ 上的一个 $[n, k]$ 随机线性码 C_2 的码字重量分布。

因为 C_2 是有限域 $\mathbb{F}_q$ 上的一个 $[n, k]$ 随机线性码, 因此, 码 C_2 中重量为 t 的码字的平均比例为

$$\frac{\binom{n}{t}(q-1)^t}{q^n}.$$

根据统计距离的定义, 可得码 C_1 和 C_2 之间的码字重量分布的统计距离为

$$\rho(|C_1|, |C_2|) = \frac{1}{2} \left(\sum_{t=1}^{n-k} \frac{\binom{n}{t}(q-1)^t}{q^n} + \sum_{t=n-k+1}^n \left| \frac{\binom{n}{t}(q-1)^t}{q^n} - \frac{N_{t-n+k}}{q^k} \right| \right).$$

□

我们取了两组参数分别计算它们的统计距离: 当 $(q, n, k) = (503, 500, 300)$ 时, $\rho(|C_1|, |C_2|) \approx 2^{-1913.4}$; 当 $(q, n, k) = (251, 250, 125)$ 时, $\rho(|C_1|, |C_2|) \approx 2^{-627.3}$ 。这表明在参数选择合适时, Reed-Solomon 码的重量分布与随机码的重量分布是不可区分的。此外, 我们能得到如下推论。

推论 6. 令 G_1 是有限域 $\mathbb{F}_q$ 上的一个 $[n, k]$ Reed-Solomon 码 C_1 的生成矩阵, G_2 和 G_3 分别是有限域 $\mathbb{F}_q$ 上两个 $[n, k]$ 随机线性码 C_2 和 C_3 的生成矩阵。令 C' 是由生成矩阵 $(G_1 + G_3 || G_3)$ 定义的线性码, C'' 是由生成矩阵 $(G_2 + G_3 || G_3)$ 定义的线性码。则码 C' 和 C'' 之间的码字重量分布的统计距离为

$$\rho(|C'|, |C''|) = \rho(|C_1|, |C_2|) = \frac{1}{2} \left(\sum_{t=1}^{n-k} \frac{\binom{n}{t}(q-1)^t}{q^n} + \sum_{t=n-k+1}^n \left| \frac{\binom{n}{t}(q-1)^t}{q^n} - \frac{N_{t-n+k}}{q^k} \right| \right).$$

又因为码 $\mathcal{C}''$ 是随机码, 所以码 $\mathcal{C}'$ 与相同长度和维数的随机码之间的码字重量分布的统计距离为

$$\frac{1}{2} \left(\sum_{t=1}^{n-k} \frac{\binom{n}{t} (q-1)^t}{q^n} + \sum_{t=n-k+1}^n \left| \frac{\binom{n}{t} (q-1)^t}{q^n} - \frac{N_{t-n+k}}{q^k} \right| \right).$$

因此, 当参数选择合适时, 我们可将 UVW 加密方案中的 $(U+V, U+W)$ 码的码字重量分布等同于由三个随机矩阵构成的 $(U+V, U+W)$ 码的码字重量分布, 其中 $\mathbf{G}_V - \mathbf{G}_W$ 是 Reed-Solomon 码的生成矩阵。有了上述分析, 我们可知找到形式为 $(\mathbf{v}, \mathbf{w})\mathbf{D}$ 的码字不能被用来作为判断给定的线性码是否是广义 $(U+V, U+W)$ 码。

现在分析找到形式为 $(\mathbf{u}, \mathbf{u})\mathbf{D}$ 的码字能不能被用来作为判断给定的线性码是否是广义 $(U+V, U+W)$ 码。下面我们将矩阵 $\mathbf{G}_U, \mathbf{G}_V$ 和 $\mathbf{G}_W$ 视为随机矩阵分析其所定义的码的码字重量分布。命题 7 给出了有限域 $\mathbb{F}_q$ 上的一个广义 $[n, k]$ $(U+V, U+W)$ 码的码字重量分布。

命题 7. 令 $\mathcal{C}$ 是有限域 $\mathbb{F}_q$ 上的一个广义 $[n, k]$ $(U+V, U+W)$ 码, 其中 U 是一个随机的 $[\frac{n}{2}, k_1]$ 线性码, V 和 W 都是随机的 $[\frac{n}{2}, k_2]$ 线性码。令 $N_{(\mathbf{u}, \mathbf{u})}$ 和 $N_{(\mathbf{u}+\mathbf{v}, \mathbf{u}+\mathbf{w})}$ 分别是形式为 $(\mathbf{u}, \mathbf{u})\mathbf{D}$ 和 $(\mathbf{u}+\mathbf{v}, \mathbf{u}+\mathbf{w})\mathbf{D}$ 的码字的平均数量。当 w 为偶数时, 有

$$N_{(\mathbf{u}, \mathbf{u})} = \frac{\binom{n/2}{w/2} (q-1)^{w/2}}{q^{n/2-k_1}}, N_{(\mathbf{u}+\mathbf{v}, \mathbf{u}+\mathbf{w})} = N_{(\mathbf{u}, \mathbf{u})} + \frac{\binom{n}{w} (q-1)^w}{q^{n-k}} \left(1 - \frac{1}{q^{k_2}}\right);$$

而当 w 为奇数时, 有

$$N_{(\mathbf{u}, \mathbf{u})} = 0, N_{(\mathbf{u}+\mathbf{v}, \mathbf{u}+\mathbf{w})} = \frac{\binom{n}{w} (q-1)^w}{q^{n-k}} \left(1 - \frac{1}{q^{k_2}}\right);$$

对于敌手来说, 他想区分广义 $(U+V, U+W)$ 码与随机码的方法就是找到广义 $(U+V, U+W)$ 码中形式为 $(\mathbf{u}, \mathbf{u})\mathbf{D}$ 的码字。

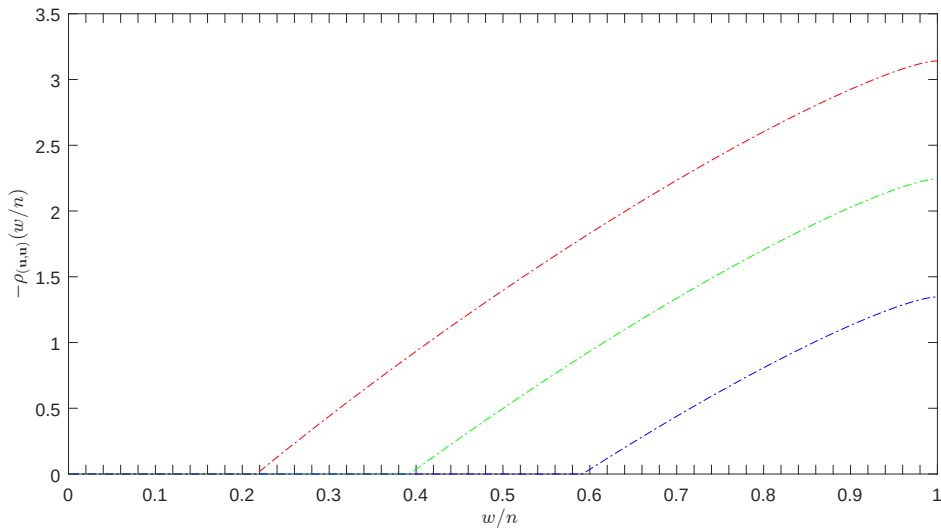


图 5: 相对重量 w/n 下形式为 $(\mathbf{u}, \mathbf{u})\mathbf{D}$ 的码字密度的归一化对数

我们在图 5 中绘制了相对重量 $x = \frac{w}{n}$ 的形式为 $(\mathbf{u}, \mathbf{u})\mathbf{D}$ 的码字密度的归一化对数图，其定义如下

$$\rho_{(\mathbf{u}, \mathbf{u})}\left(\frac{w}{n}\right) = \frac{\log_2(N_{(\mathbf{u}, \mathbf{u})}/N_{(\mathbf{u}+\mathbf{v}, \mathbf{u}+\mathbf{w})})}{n}.$$

取 $q = 503, n = 1000, k = 500$ ，图中的三条线都是在这个参数下得出的，它们只在 k_1 和 k_2 上取值不同，其中从左到右三条线分别是 k_1 取值为 $0.3k, 0.5k$ 和 $0.7k$ 时的码字密度的归一化对数曲线图。我们能看当相对重量 w/n 分别小于 $0.22, 0.4$ 和 0.59 时，几乎所有的码字都是 $(\mathbf{u}, \mathbf{u})\mathbf{D}$ 的形式。对于我们选择的参数 $(q, n, k) = (503, 1000, 500)$ ，找到一个重量在区间 $[\frac{q-1}{q}(n-k)+1, k+\frac{q-1}{q}(n-k)] = [500.006, 999.006]$ 之间的码字是简单的。因此，当 $k_1 = 0.7k$ 时，敌手能够很轻松分辨给定的码是否是广义 $(U+V, U+W)$ 码，而当 $k_1 = 0.3k, 0.5k$ 时，敌手需要指数时间才能分辨给定的码是否是广义 $(U+V, U+W)$ 码。

前面已经证明了只要码区分问题和一般译码问题计算困难，则 UVW 公钥加密方案就是 IND-CPA 安全的，又由于码区分问题最后还是归结到了固定码字重量寻找问题，因此我们能够利用信息集译码算法为 UVW 公钥加密方案给出保守的安全参数估计。

信息集译码算法最早由 Prange [6] 于 1962 年引入。它的概念涉及识别一个信息集，并猜测信息集中的坐标是不发生错误的。1988 年，Lee 和 Brickell [7] 通过假设信息集也可能包含一些错误来改进 Prange 的 ISD 算法。同年晚些时候，Leon [8] 和 Stern [9] 进一步改进了 Lee 和 Brickell 提出的算法。随后，几乎所有的增强算法都结合了 Lee-Brickell 和 Stern 的中间相遇策略的原理。ISD 算法已经有了许多改进和分析，例如 [10, 11, 12, 13, 14, 15] 等。

2010 年，Peters [16] 将 Lee-Brickell [7] 和 Stern [9] 的信息集译码算法从二元域推广到任意有限域 $\mathbb{F}_q$ ，这是第一个 q 元有限域上的信息集译码算法，但该算法的计算复杂度与 q 的大小相关，当 q 越大时，效率越低。2017 年，Niebuhr 等人 [17] 借鉴 Finiasz-Sendrier [18] 中的思想，对 Peters [16] 中的信息集译码算法进一步优化，提出并证明了有限域 $\mathbb{F}_q$ 上信息集译码算法复杂性的下限。该算法代表了一种理想化的信息集译码类算法，这意味着只考虑最关键步骤的成本，而不考虑开销、内存访问等成本，因此常被用于为基于编码的方案给出保守的安全参数估计。这里我们就不对 Niebuhr 等人 [17] 的算法做详细的描述了，感兴趣的读者可以自行查阅文献。下面我们直接给出使用 Niebuhr 等人 [17] 的算法寻找重量为 w 的码字的预期计算工作量：

命题 8. 令 $\mathbf{H} \in \mathbb{F}_q^{(n-k) \times n}$ 是随机矩阵， $\mathbf{s} \in \mathbb{F}_q^{n-k}$ 是随机给定的校验子。如果 $\binom{n}{w}(q-1)^w \geq q^{n-k}$ (多个解) 且 $\binom{n-k}{w-v}\binom{k}{v}(q-1)^w \ll q^{n-k}$ ，则调用文献 [17] 中的算法求解校验子译码问题实例 $SD(\mathbf{H}, \mathbf{s}, w)$ 的平均译码复杂度为

$$\begin{aligned} & WF_{qISD}(q, n, k, w) \\ &= \min_{l, v_1, v_2} \frac{1}{\sqrt{q-1}} \frac{q^{n-k}}{\binom{n-k-l}{w-v}\binom{k}{v_1}\binom{l}{v_2}(q-1)^w} \left(K_q \frac{\binom{k}{v_1}\binom{l}{v_2}(q-1)^{v-1}}{q^l} \right. \\ & \quad \left. + \lambda_q^{-1} \left(\frac{2(q-1)l}{q\binom{l}{v'_2}(q-1)^{v'_2}} + v_2 \right) \times \sqrt{\binom{k}{v_1}\binom{l}{v_2}(q-1)^{v-1}} \right), \end{aligned} \quad (21)$$

其中

$$\begin{aligned} v &= v_1 + v_2, \quad v'_2 = \lfloor v_2/2 \rfloor, \\ \lambda_q &= 1 - \exp(-1) \approx 0.63, \quad K_q = 2w. \end{aligned}$$

这里的 l 、 v_1 和 v_2 都是可以自由选择的优化参数。

如果 $\mathbf{s} = \mathbf{eH}^\top$ ，且 $wt(\mathbf{e}) = w$ 、 $\binom{n}{w}(q-1)^w < q^{n-k}$ (一个解)、 $\binom{n-k}{w-v}\binom{k}{v}(q-1)^w \ll q^{n-k}$ ，则调用文献 [17] 中的算法求解校验子译码问题实例 $SD(\mathbf{H}, \mathbf{s}, w)$ 的平均译码复杂度为

$$\begin{aligned} WF_{qISD}(q, n, k, w) &= \min_{l, v_1, v_2} \frac{1}{\sqrt{q-1}} \frac{\binom{n}{w}(q-1)^w}{\binom{n-k-l}{w-v}\binom{k}{v_1}\binom{l}{v_2}(q-1)^w} \left(K_q \frac{\binom{k}{v_1}\binom{l}{v_2}(q-1)^{v-1}}{q^l} \right. \\ &\quad \left. + \lambda_q^{-1} \left(\frac{2(q-1)l}{q\binom{l}{v_2}(q-1)^{v_2'}} + v_2 \right) \times \sqrt{\binom{k}{v_1}\binom{l}{v_2}(q-1)^{v-1}} \right). \end{aligned} \quad (22)$$

例 9. 在 UVW128 对应的参数集下, $(q, n, k, w) = (433, 860, 430, 116)$, 由于 $\binom{n}{w}(q-1)^w \ll q^{n-k}$ (一个解), 因此计算

$$\begin{aligned} WF_{qISD}(q, n, k, w) &= \min_{l, v_1, v_2} \frac{1}{\sqrt{q-1}} \frac{\binom{n}{w}(q-1)^w}{\binom{n-k-l}{w-v}\binom{k}{v_1}\binom{l}{v_2}(q-1)^w} \left(K_q \frac{\binom{k}{v_1}\binom{l}{v_2}(q-1)^{v-1}}{q^l} \right. \\ &\quad \left. + \lambda_q^{-1} \left(\frac{2(q-1)l}{q\binom{l}{v_2}(q-1)^{v_2'}} + v_2 \right) \times \sqrt{\binom{k}{v_1}\binom{l}{v_2}(q-1)^{v-1}} \right) \\ &\approx 2^{128.3613}. \end{aligned}$$

由例 9 可以看出 UVW128 在 ISD 算法攻击下达到了 128 比特经典安全强度级别。类似的分析, 在 UVW256 和 UVW512 对应的参数下, ISD 算法攻击复杂度分别为 $2^{256.8354}$ 和 $2^{513.2195}$, 即这两组参数也达到了对应的安全级别。

前面已经证明了具有参数 k_1, k_2 的一个 $[n, k = k_1 + k_2]$ ($U+V, U+W$) 码 $\mathcal{C}$ 的对偶码 $\mathcal{C}^\perp$ 是一个具有参数 $n/2 - k_2, n/2 - k_1$ 的 $[n, n-k]$ ($U+V, U+W$) 码, 即码 $\mathcal{C}$ 中形式为 $(\mathbf{u}, \mathbf{u})\mathbf{D}$ 的码字的维数为 k_1 , 而码 $\mathcal{C}^\perp$ 中形式为 $(\mathbf{u}, \mathbf{u})\mathbf{D}$ 的码字的维数为 $n/2 - k_2$ 。如果 $k_1 \neq n/2 - k_2$, 那么选择的参数必须同时考虑到码 $\mathcal{C}$ 及其对偶码 $\mathcal{C}^\perp$ 在区分问题上的计算复杂度。而如果 $k_1 = n/2 - k_2$, 则码 $\mathcal{C}$ 及其对偶码 $\mathcal{C}^\perp$ 的区分问题是相同的。为了简单起见, 因此我们选择的参数满足 $k_1 = n/2 - k_2$, 这样我们只需考虑单个区分问题的计算复杂度。

从命题 7 能够看出, 随着 w 的增大, $(\mathbf{u}, \mathbf{u})\mathbf{D}$ 形式的码字的数量越来越多, 但是其所占的比例越来越少。对于信息集译码来说, 满足条件的解的数量越多, 计算复杂度越小。因此我们取如下码字重量参数

$$t_{max} = \max_w \left\{ w \mid \frac{\binom{n/2}{w/2}(q-1)^{w/2}}{q^{n/2-k_1}} > \frac{\binom{n}{w}(q-1)^w}{q^{n-k}} \left(1 - \frac{1}{q^{k_2}} \right) \right\} \quad (23)$$

作为计算区分问题的计算复杂度的码字重量参数。

另一方面, 要寻找重量为 t_{max} 的码字, 首先得确保码 $\mathcal{C}$ 中存在这样的码字。令

$$t_{min} = \min_w \left\{ w \mid \frac{\binom{n/2}{w/2}(q-1)^{w/2}}{q^{n/2-k_1}} \geq 1 \right\}. \quad (24)$$

那么只要 $t_{max} \geq t_{min}$, 码 $\mathcal{C}$ 中就存在重量为 t_{max} 的码字。

令

$$\begin{aligned} r_{t_{min}} &= \frac{\binom{n/2}{t_{min}/2}(q-1)^{t_{min}/2}}{q^{n/2-k_1}} \bigg/ \frac{\binom{n}{t_{min}}(q-1)^{t_{min}}}{q^{n-k}} \\ &= \frac{\binom{n/2}{t_{min}/2}q^{n/2-k_2}}{\binom{n}{t_{min}}(q-1)^{t_{min}/2}} \\ &\leq \frac{q^{n/2-k_2}}{\binom{n/2}{t_{min}/2}(q-1)^{t_{min}/2}}. \end{aligned} \quad (25)$$

如果 $k_1 \leq k_2$, 则

$$r_{t_{min}} \leq \frac{q^{n/2-k_2}}{\binom{n/2}{t_{min}/2}(q-1)^{t_{min}/2}} \leq \frac{q^{n/2-k_1}}{\binom{n/2}{t_{min}/2}(q-1)^{t_{min}/2}} \leq 1, \quad (26)$$

即 $t_{max} \leq t_{min}$ 。也就是说, 对任意的 $t \geq t_{min}$, 有

$$\frac{\binom{n/2}{t/2}(q-1)^{t/2}}{q^{n/2-k_1}} \leq \frac{\binom{n}{t}(q-1)^t}{q^{n-k}} \left(1 - \frac{1}{q^{k_2}}\right). \quad (27)$$

因此, $k_1 \leq k_2$ 时, 形式为 $(\mathbf{u}, \mathbf{u})\mathbf{D}$ 码字的期望数量比非 $(\mathbf{u}, \mathbf{u})\mathbf{D}$ 形式的码字的期望数量少。

令 E_1 表示在重量为 t 时 $(\mathbf{u}, \mathbf{u})\mathbf{D}$ 码字的期望数量, E_2 表示重量为 t 时非 $(\mathbf{u}, \mathbf{u})\mathbf{D}$ 码字的期望数量:

$$E_1 = \frac{\binom{n/2}{t/2}(q-1)^{t/2}}{q^{n/2-k_1}}, E_2 = \frac{\binom{n}{t}(q-1)^t}{q^{n-k}} \left(1 - \frac{1}{q^{k_2}}\right).$$

下面我们分析 UVW128 对应参数下, E_1 和 E_2 的取值。

由图 6 我们可以看到, 当 $k_1 = k_2$ 时, $(\mathbf{u}, \mathbf{u})\mathbf{D}$ 形式的码字与非 $(\mathbf{u}, \mathbf{u})\mathbf{D}$ 形式的码字几乎同时出现, 并且两者的期望数量关系满足式子 (27), 随着 t 的增大, 这两种形式的码字期望数量差距越来越大。在 UVW128 对应参数下, 找一个重量在区间 $[430, 859]$ 之间的码字是简单的, 但是当 $t = 430$ 时, E_2 就已经大约是 E_1 的 2^{428} 倍, 所以在 $t \geq 430$ 的区间内找到一个码字是 $(\mathbf{u}, \mathbf{u})\mathbf{D}$ 形式的概率是可忽略的。由于区分广义 $(U+V, U+W)$ 码与随机码的方法就是找到广义 $(U+V, U+W)$ 码中形式为 $(\mathbf{u}, \mathbf{u})\mathbf{D}$ 的码字, 因而我们想分析两者期望数量差距不太大时码区分问题的困难性。图 7 中, 我们具体分析了 $t \in [330, 430]$ 时, E_1 与 E_2 的期望数量关系。当 $t = 336$ 时, 两种形式的码字开始出现, 且期望数量差距最小, 在此处调用 ISD 算法找一个码字的计算复杂度大约为 $2^{502.6478}$ 。当 $t = 430$ 时, 调用 ISD 算法找一个码字是容易的, 但是 E_1 和 E_2 期望数量差距过大, 几乎找不到 $(\mathbf{u}, \mathbf{u})\mathbf{D}$ 形式的码字。

令 $F(t)$ 表示调用 ISD 算法在重量为 t 处找一个码字的计算复杂度, $P(t) \approx \frac{E_1}{E_1+E_2}$ 表示在重量 t 处找到一个码字是 $(\mathbf{u}, \mathbf{u})\mathbf{D}$ 形式的概率, 则在重量 t 处找到 $(\mathbf{u}, \mathbf{u})\mathbf{D}$ 形式码字的计算复杂度为 $F_{(\mathbf{u}, \mathbf{u})\mathbf{D}}(t) = \frac{F(t)}{P(t)}$, 我们以此作为码区分问题困难性的衡量标准。经遍历分析, 在目标区间 $t \in [336, 430]$ 内, 码区分问题的总攻击复杂度 $F_{(\mathbf{u}, \mathbf{u})\mathbf{D}}(t)$ 的最小值为 $2^{403.71}$ (出现于 $t = 410$), 该值远大于 128-bit 安全性下界, 当 $t > 430$ 时, 总复杂度完全由指数级增长的码字期望数量差距主导, 因此必然高于区间 $[336, 430]$ 内的最小值。因此 UVW128 对应的参数满足安全要求。类似的分析, UVW256 和 UVW512 的码区分问题的总攻击复杂度 $F_{(\mathbf{u}, \mathbf{u})\mathbf{D}}(t)$ 的最小值分别为 $2^{815.35}$ (出现于 $t = 826$) 和 $2^{1649.78}$ (出现于 $t = 1666$), 都超过了对应的安全性下界, 即 UVW256 和 UVW512 也达到了对应的安全级别。

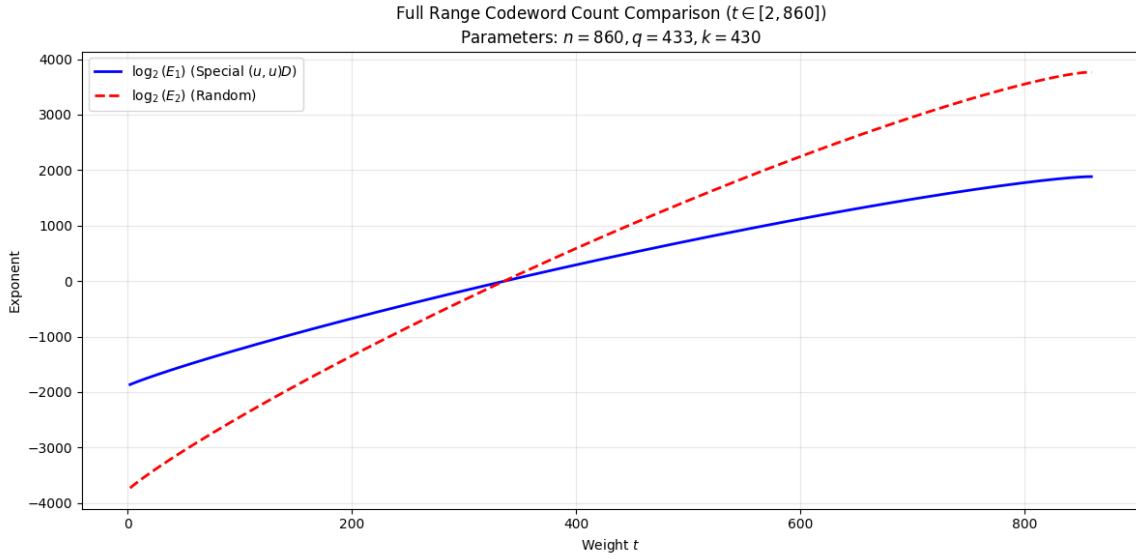


图 6: 重量为 t 时 $(u, u)D$ 码字的期望数量与非 $(u, u)D$ 形式码字的期望数量对比

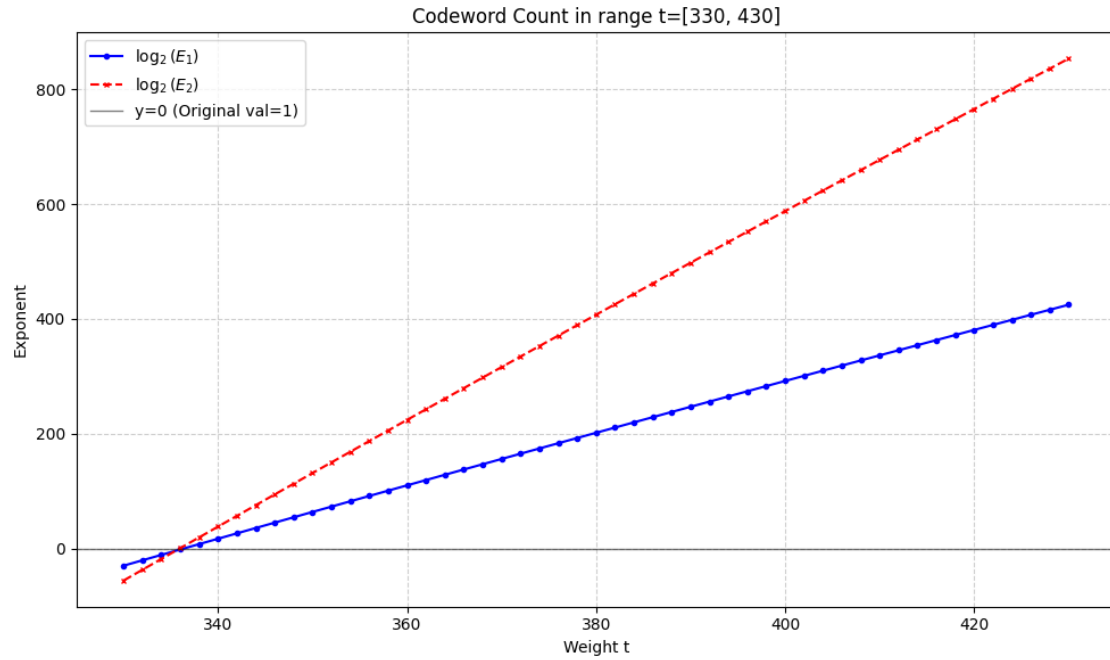


图 7: 重量 t 在 $[330, 430]$ 时 $(u, u)D$ 码字的期望数量与非 $(u, u)D$ 形式码字的期望数量对比

3.2.2 量子安全强度分析

为评估 UVW 方案在量子攻击下的安全性, 我们参考 Esser 与 Bellini 提出的保守量子复杂度估计框架 [19]。该框架在 NIST 所定义的量子电路最大深度约束 (`maxdepth`) 下, 分析 Grover 加速的 Prange-ISD 算法, 并给出了针对校验子译码问题 (SD) 的量子门复杂度估计。目前已知的针对随机线性码校验子译码问题的最佳通用攻击仍属于 ISD 类算法, 其中主要的指数复杂度来自对合适信息集 (或等价置换) 的搜索过程。在量子情形下, 该搜索过程可以通过 Grover 算法获得平方根级别的加速。

需要指出的是, [19] 中的量子复杂度估计主要针对二元域 $\mathbb{F}_2$ 上的随机线性码校验子译码问题。然而, 该框架的核心思想是将 Prange-ISD 中对置换 (信息集) 的搜索过程视为无结构搜索, 并在 NIST 规定的电路深度约束下采用 Grover 搜索及其分块策略进行量子复杂度估计。该分析在形式上仅依赖于一次尝试成功的概率以及单次验证电路的深度, 因此可自然用于 $\mathbb{F}_q$ 上的情形。

在本文中, 我们将该估计方法用于 UVW 方案所对应的 q 元线性码 SD 问题, 并采用与二元情形一致的组合概率来近似信息集选择的成功概率。同时, 我们沿用文献 [19] 中对高斯消元深度的乐观估计。由于在 $\mathbb{F}_q$ 上实现线性代数运算的量子电路深度通常不小于 $\mathbb{F}_2$, 因此该做法相当于对攻击者能力的乐观假设, 从而给出了对方案量子安全性的保守估计。

在 Prange-ISD 中, 一次随机置换成功的概率主要取决于错误支持集是否落在冗余坐标集合中。按照 [19] 的处理方式, 我们采用二元情形下的组合概率作为近似:

$$p_{\text{succ}} := \frac{\binom{n-k}{w}}{\binom{n}{w}}. \quad (28)$$

令 D_E 表示一次 ISD 尝试中高斯消元与验证过程所需的量子电路深度。根据 [19] 有,

$$D_E := (n - k)^{\omega_M}, \quad (29)$$

其中 $\omega_M \in [2, 3]$ 为矩阵乘法常数。本文与 [19] 保持一致, 取 $\omega_M = 2.5$ 。需要注意的是, 在 $\mathbb{F}_q$ 上执行消元与乘法运算的实际电路深度通常大于 $\mathbb{F}_2$, 因此该深度模型对攻击者而言是乐观的, 从防守方角度看属于保守估计。

通过利用 Grover 搜索, 使用一次量子电路应用来找到一个好的置换, 其所需的电路深度为,

$$D = O\left(D_E \sqrt{p_{\text{succ}}^{-1}}\right). \quad (30)$$

然而, NIST 在量子安全定义中引入最大电路深度约束 `maxdepth`, 规定单个量子电路的深度不得超过

$$D_{\text{max}} := 2^{\text{maxdepth}}. \quad (31)$$

根据 Zalka 对 Grover 搜索最优性的分析 [20], 当 D 超过该约束时, 必须将搜索空间划分为 N 个等大小分区, 使得每个分区上执行一次 Grover 搜索的电路深度不超过限制。此时单次分区搜索的深度为

$$D_N = O\left(D_E \sqrt{(p_{\text{succ}} N)^{-1}}\right). \quad (32)$$

令 $D_N = 2^{\text{maxdepth}}$ 可得

$$N = \frac{(D_E)^2 \cdot p_{\text{succ}}^{-1}}{2^{2 \cdot \text{maxdepth}}}. \quad (33)$$

由于该深度为 2^{maxdepth} 的电路需重复运行 N 次, 故总量子门复杂度估计为

$$T_Q = N \cdot 2^{\text{maxdepth}} = \frac{(D_E)^2}{p_{\text{succ}} \cdot 2^{\text{maxdepth}}}. \quad (34)$$

将式 (28) 与式 (29) 代入式 (34), 得到显式形式

$$T_Q = (n - k)^{2\omega_M} \cdot \frac{\binom{n}{w}}{\binom{n-k}{w}} \cdot 2^{-\text{maxdepth}}. \quad (35)$$

NIST 采用在相同 maxdepth 约束下破解 AES 的量子门复杂度作为基准, 并定义:

$$\begin{aligned} T_{\text{AES128}} &= 2^{170 - \text{maxdepth}}, \\ T_{\text{AES192}} &= 2^{233 - \text{maxdepth}}, \\ T_{\text{AES256}} &= 2^{298 - \text{maxdepth}}. \end{aligned}$$

因此, 若某参数集满足

$$\log_2(T_Q) \geq \lambda_\ell - \text{maxdepth},$$

则称其达到 NIST 量子安全等级 $\ell \in \{1, 3, 5\}$, 其中 $\lambda_1 = 170$ (对应 AES-128), $\lambda_3 = 233$ (对应 AES-192), $\lambda_5 = 298$ (对应 AES-256)。

例 10. 在 $UVW128$ 对应的参数集下, $(n, k, w) = (860, 430, 116)$, 取 $\omega_M = 2.5$, 计算

$$\begin{aligned} \log_2(T_Q) &= 172.72 - \text{maxdepth} \\ &\geq 170 - \text{maxdepth} \end{aligned}$$

由例 10 可以看出 $UVW128$ 达到了 128 比特量子安全强度级别。类似的分析, 在 $UVW256$ 对应的参数下 $\log_2(T_Q) = 306.98 - \text{maxdepth} \geq 170 - \text{maxdepth}$, 在 $UVW512$ 对应的参数下 $\log_2(T_Q) = 569.21 - \text{maxdepth} \geq 298 - \text{maxdepth}$, 即两组参数分别达到了 128 比特和 256 比特量子安全强度级别。

4 性能评估

4.1 理论性能分析

UVW 方案的计算开销主要集中在以下三个模块:

有限域运算 本方案在 $\mathbb{F}_q$ 上运算, 求逆通过预计算查找表实现。特别地, 加法/减法采用无分支的“加减掩码修正”技术, 有效避免了条件跳转带来的时序差异。向量矩阵乘法通过 32-bit 累加器延迟取模, 显著降低了模运算开销。

高斯消元 密钥生成阶段需要对 $k \times n$ 矩阵进行高斯消元, 复杂度为 $O(k^2 \cdot n)$ 。实现中利用了消元过程中左下角零元素的特性, 减少了不必要的运算。随着安全级别升高, k 和 n 成倍增长, 消元时间是密钥生成的主要开销。

RS 列表译码 解密阶段的核心是 Guruswami-Sudan 列表译码算法, 其复杂度主要由插值步和因子分解步决定。实际运行中, 译码占据了绝大部分解密时间。

4.2 性能测试

性能测试平台 性能测试在一台运行 Windows 11 专业版 (64 位) 的机器上进行, 该机器拥有 128 GB 内存和 Intel® Core™ i9-14900K CPU @ 3.20 GHz。对于每个参数集, 分别独立执行 1000 次密钥生成、1000 次封装和 1000 次解封装操作。每次封装使用随机生成的密钥对, 每次解封装使用与之对应的密文。记录每次操作的 CPU 周期。将 1000 个密钥生成周期、1000 个封装周期和 1000 个解封装周期分别排序, 给出下四分位数、中位数、平均值和上四分位数, 同时给出密钥生成、密钥封装和密钥解封装 1000 次操作在本测试平台下的平均运行时间。

传输与存储开销 表 2 给出了不同安全级别推荐参数下公钥、私钥、密文和会话密钥的尺寸。

参考实现 参考实现的性能数据在表 3 中给出。参考实现以 C99 编写, 使用 gcc (MinGWw64, 15.1.0) 编译, 编译选项为 -O3 -std=c99 -flto -funroll-all-loops -Wall -Wextra -pedantic。

优化实现 优化实现的性能数据在表 4 中给出。优化实现以 C99 编写, 利用了 AVX2 指令, 使用 gcc (MinGWw64, 15.1.0) 编译, 编译选项为 -O3 -std=c99 -flto -funroll-all-loops -march=native -Wall -Wextra -pedantic。

5 特点声明

- **算法的创新性。**本方案提出了广义 $(U+V, U+W)$ 码 [21] 这一新型线性码结构, 将其用于构造基于编码的密钥封装机制。与传统 McEliece 型方案不同, UVW 在私钥拼装成的广义 $(U+V, U+W)$ 码嵌入了 Reed-Solomon 码的生成矩阵作为陷门, 使得合法解密者可通过列表译码和线性方程组求解高效恢复明文, 而敌手在公钥视角下面面对的是与随机线性码不可区分的问题实例。与已有的基于 $(U, U+V)$ 码构造的公钥加密方案 [22] 相比, UVW 选择广义 $(U+V, U+W)$ 码构造, 生成的公钥矩阵具有更多的可能性, 进而能够使得公钥矩阵与随机矩阵更加难以区分。
- **算法的存储开销优势。**以 NIST 标准化进程中的 Classic McEliece 方案为参照, UVW 在公钥和私钥尺寸上具有突出的竞争力。前者在 128 bit 和 256 bit 经典安全级别下的公钥尺寸介于 255 KB 至 1326 KB 之间, 私钥尺寸约为 6.3 KB 至 13.6 KB。而 UVW 在相同经典安全级别下公钥尺寸分别为 203.1 KB 和 890.2 KB, 私钥尺寸则通过种子派生策略压缩为固定 35 字节 (128/256 bit 级别) 和 67 字节 (512 bit 级别), 远小于 Classic McEliece 的私钥存储开销。

表 2: UVW 密钥封装机制传输与存储开销 (单位: Bytes)

	Public key	Private key	Ciphertext	Session key
UVW128	208013	35	1032	64
UVW256	911645	35	2199	64
UVW512	4001850	67	4820	64

表 3: UVW 密钥封装机制参考实现性能测试结果

	operation	quartile	median	average	quartile	time
UVW128	KeyGen	172957580	174424628	175360080	176185729	54ms
UVW256	KeyGen	1452749029	1462650837	1469615517	1472240134	459ms
UVW512	KeyGen	11373282624	11426363913	11437841258	11503468492	3574ms
UVW128	Encaps	414319	416606	470955	420866	0.14ms
UVW256	Encaps	1594847	1639596	1702005	1703332	0.53ms
UVW512	Encaps	11463351	11644865	11761738	11889606	3.67ms
UVW128	Decaps	414038349	426662590	428882166	439507552	134ms
UVW256	Decaps	3448351931	3523949348	3545934587	3607094531	1108ms
UVW512	Decaps	20007622666	20290008773	20483434568	20613873389	6401ms

表 4: UVW 密钥封装机制优化实现性能测试结果

	operation	quartile	median	average	quartile	time
UVW128	KeyGen	82168509	82981857	83929312	84066306	26ms
UVW256	KeyGen	586859085	592341076	599684897	598800552	187ms
UVW512	KeyGen	4480146633	4506464435	4517173867	4536788037	1411ms
UVW128	Encaps	417765	421225	458819	425419	0.14ms
UVW256	Encaps	1608653	1658872	1704923	1702592	0.53ms
UVW512	Encaps	11282320	11478497	11777750	11947496	3.68ms
UVW128	Decaps	302732336	317785567	319782835	332492629	100ms
UVW256	Decaps	2616173122	2705466876	2719331496	2799935508	849ms
UVW512	Decaps	12190815341	12495559600	12607905533	12912025565	3939ms

参考文献

- [1] Venkatesan Guruswami and Madhu Sudan. “Improved decoding of Reed-Solomon and algebraic-geometric codes”. In: *Proceedings 39th Annual Symposium on Foundations of Computer Science (Cat. No. 98CB36280)*. IEEE. 1998, pp. 28–37.
- [2] Mingsheng Wang. “Parameter choices on Guruswami-Sudan algorithm for polynomial reconstruction”. In: *Finite Fields Their Appl.* 13.4 (2007), pp. 877–886.
- [3] Zhe-xian Wan. “Geometry of classical groups over finite fields and its applications”. In: *Discrete Mathematics* 174.1-3 (1997), pp. 365–381.
- [4] Dennis Hofheinz, Kathrin Hövelmanns, and Eike Kiltz. “A Modular Analysis of the Fujisaki-Okamoto Transformation”. In: *Theory of Cryptography, TCC 2017*. Ed. by Yael Kalai and Leonid Reyzin. Cham: Springer, 2017, pp. 341–371. ISBN: 978-3-319-70500-2.
- [5] M. Naor and M. Yung. “Public-Key Cryptosystems Provably Secure against Chosen Ciphertext Attacks”. In: *22nd Annual ACM Symposium on Theory of Computing, STOC 1990*. ACM, 1990, pp. 427–437. ISBN: 0897913612.
- [6] Eugene Prange. “The use of information sets in decoding cyclic codes”. In: *IRE Trans. Inf. Theory* 8.5 (1962), pp. 5–9.
- [7] Pil Joong Lee and Ernest F. Brickell. “An Observation on the Security of McEliece’s Public-Key Cryptosystem”. In: *EUROCRYPT 1988*. Ed. by Christoph G. Günther. Vol. 330. LNCS. Springer, 1988, pp. 275–280.
- [8] Jeffrey S. Leon. “A probabilistic algorithm for computing minimum weights of large error-correcting codes”. In: *IEEE Trans. Inf. Theory* 34.5 (1988), pp. 1354–1359.
- [9] Jacques Stern. “A method for finding codewords of small weight”. In: *Coding Theory and Applications, 3rd International Colloquium, Toulon, France, November 2-4, 1988, Proceedings*. Ed. by Gérard D. Cohen and Jacques Wolfmann. Vol. 388. LNCS. Springer, 1988, pp. 106–113.
- [10] Daniel J. Bernstein, Tanja Lange, and Christiane Peters. “Smaller Decoding Exponents: Ball-Collision Decoding”. In: *CRYPTO 2011*. Ed. by Phillip Rogaway. Vol. 6841. LNCS. Springer, 2011, pp. 743–760.
- [11] Anja Becker et al. “Decoding Random Binary Linear Codes in $2^{n/20}$: How $1 + 1 = 0$ Improves Information Set Decoding”. In: *EUROCRYPT 2012*. Ed. by David Pointcheval and Thomas Johansson. Vol. 7237. LNCS. Springer, 2012, pp. 520–536.
- [12] Alexander May, Alexander Meurer, and Enrico Thomae. “Decoding Random Linear Codes in $\mathcal{O}(2^{0.054n})$ ”. In: *ASIACRYPT 2011*. Ed. by Dong Hoon Lee and Xiaoyun Wang. Vol. 7073. LNCS. Springer, 2011, pp. 107–124.
- [13] Alexander May and Ilya Ozerov. “On Computing Nearest Neighbors with Applications to Decoding of Binary Linear Codes”. In: *EUROCRYPT 2015*. Ed. by Elisabeth Oswald and Marc Fischlin. Vol. 9056. LNCS. Springer, 2015, pp. 203–228.

-
- [14] Marco Baldi et al. “A Finite Regime Analysis of Information Set Decoding Algorithms”. In: *Algorithms* 12.10 (2019), p. 209.
 - [15] Norman Lahr et al. “Side Channel Information Set Decoding Using Iterative Chunking - Plaintext Recovery from the "Classic McEliece" Hardware Reference Implementation”. In: *ASIACRYPT 2020*. Ed. by Shiho Moriai and Huaxiong Wang. Vol. 12491. LNCS. Springer, 2020, pp. 881–910.
 - [16] Christiane Peters. “Information-set decoding for linear codes over $\mathbb{F}_q$ ”. In: *Post-Quantum Cryptography - PQC 2010*. Ed. by Nicolas Sendrier. Berlin, Heidelberg: Springer, 2010, pp. 81–94. ISBN: 978-3-642-12929-2.
 - [17] Robert Niebuhr et al. “On lower bounds for information set decoding over $\mathbb{F}_q$ and on the effect of partial knowledge”. In: *Int. J. Inf. Coding Theory* 4.1 (2017), pp. 47–78.
 - [18] Matthieu Finiasz and Nicolas Sendrier. “Security Bounds for the Design of Code-Based Cryptosystems”. In: *ASIACRYPT 2009*. Ed. by Mitsuru Matsui. Berlin, Heidelberg: Springer, 2009, pp. 88–105.
 - [19] Andre Esser and Emanuele Bellini. “Syndrome Decoding Estimator”. In: *Public Key Cryptography (1)*. Lecture Notes in Computer Science. Springer, 2022, pp. 112–141.
 - [20] Christof Zalka. “Grover’ s quantum searching algorithm is optimal”. In: *Physical Review A* 60.4 (1999), p. 2746.
 - [21] Yang Yang and Fangguo Zhang. “An Efficient Encryption Scheme Based on $(U+V, U+W)$ Codes”. In: *IACR Cryptol. ePrint Arch.* 2025 (2025), p. 1149.
 - [22] Irene Marquez Corbella and Jean-Pierre Tillich. “Using Reed-Solomon codes in the $(U \mid U + V)$ construction and an application to cryptography”. In: *ISIT*. IEEE, 2016, pp. 930–934.